CSE221: Algorithms

Graphs

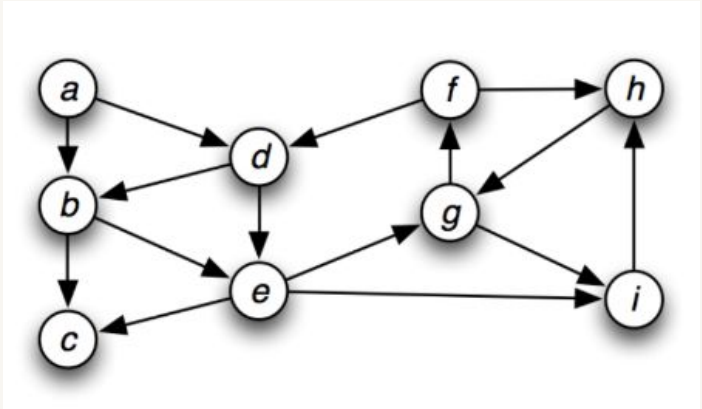
Prepared by: A B M Muntasir Rahman [MUNR]



What is a Graph?

- Graph G is pair (V, E) where V is a finite set (set of vertices) and E is a finite set of pairs from V (set of edges).
- We generally denote n = |V| and m = |E|.

Here,
V = {a, b, …, i} and n = |V| = 9
E = {ab, bc, ad, …..} and m = |E| = 15

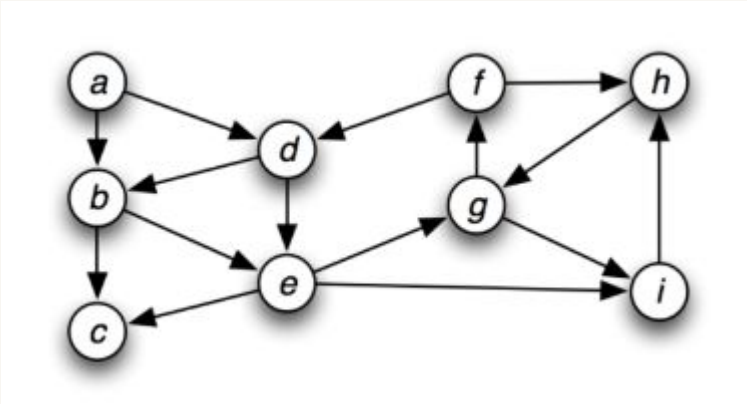


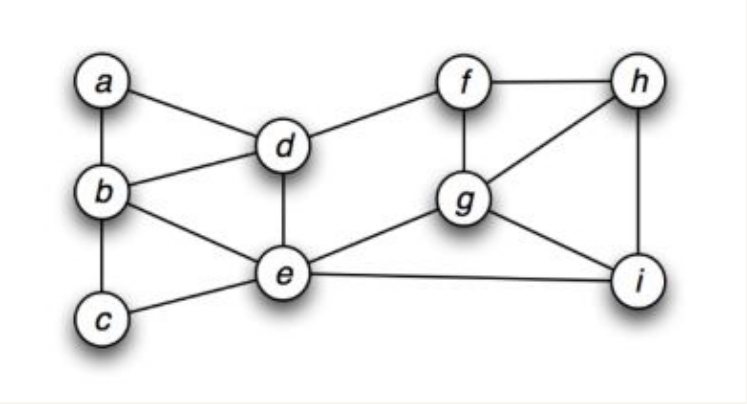
Applications of Graphs

- **Transportation Networks:** How should you design the roads and highway network in a country? What is the quickest way to go from BRAC University to Mirpur?
- **Communication Networks:** How to send network packet from your computer to your friend's computer?
- **Social Networks:** Facebook, Instagram, Twitter etc.
- **Dependency Networks:** What courses you must take before you can take CSE470?

Directed vs Undirected Graphs

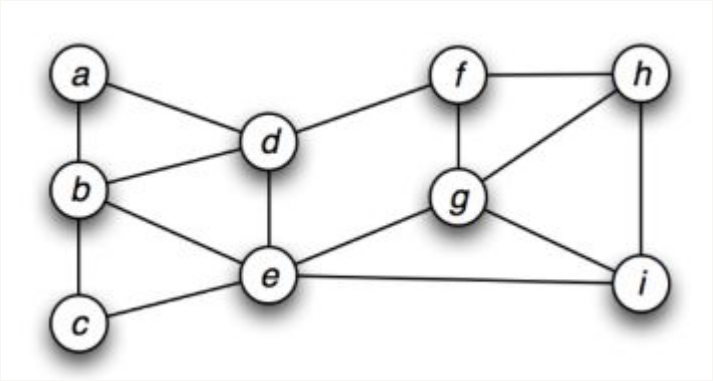
- Graph G can be **directed,** if E consists of ordered pairs, or **undirected**, if E consists of unordered pairs.
- If (u, v) Є E, then vertices u and v are adjacent.

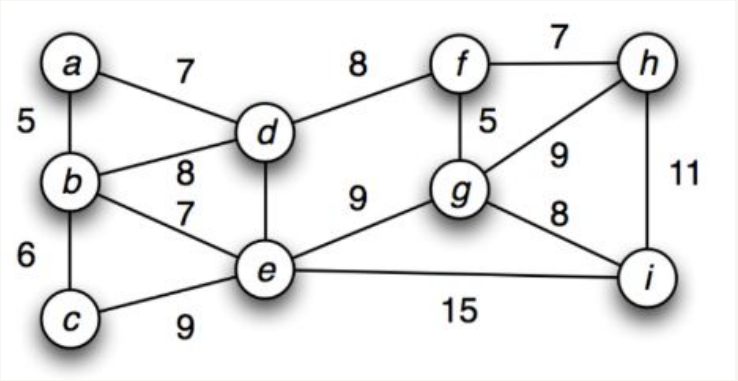




Weighted vs Unweighted Graphs

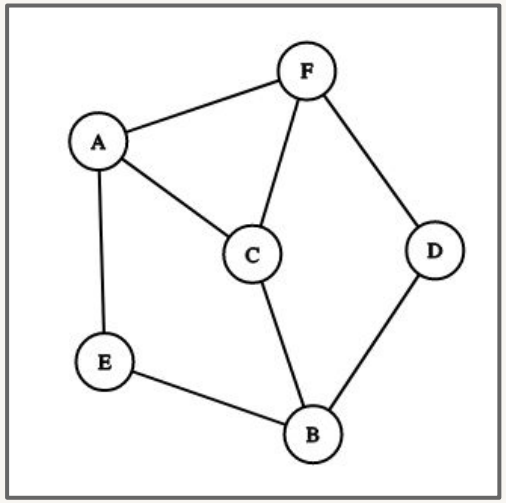
- Weights can be assigned to the edges i.e. **w_G (e) is a weight of the edge e Є E**. Any graph that has such function assigned to it's edges is called **weighted graph**.





Basic Terminologies

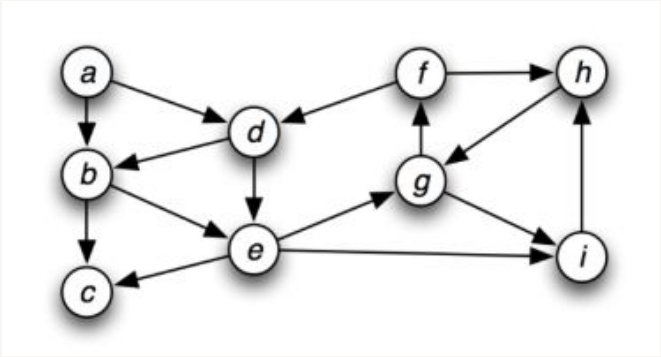
- **Edges:** Connections/Links between vertices (or nodes).



Here, AF, AC, BC etc. are edges of the graph.

Basic Terminologies

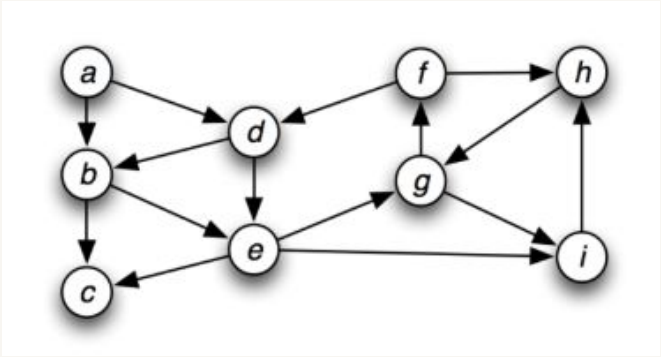
- **Incoming Edges:** Directed edges coming into a vertex.



Here, AB and DB are two incoming edges of vertex B.

Basic Terminologies

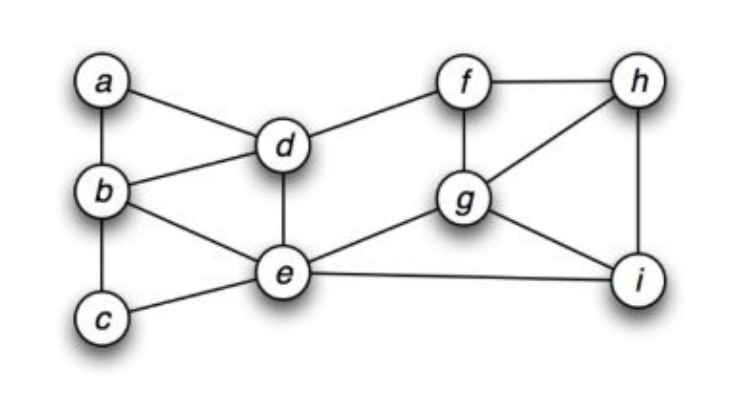
- **Outgoing Edges:** Directed edges going out of a vertex.



Here, BC and BE are two outgoing edges of vertex B.

Basic Terminologies

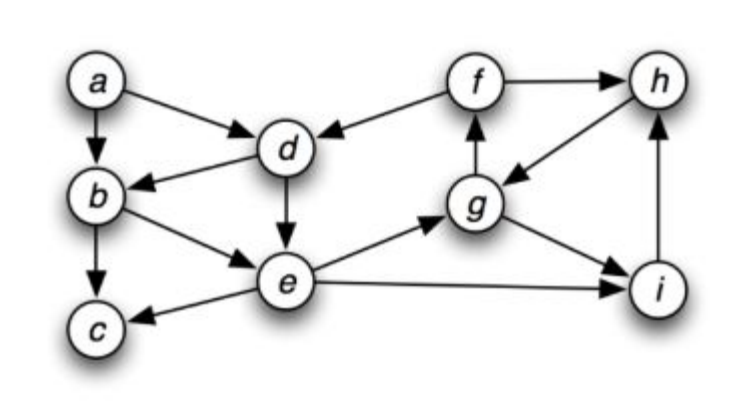
- **Degree:** For an undirected graph, **number of connected edges** to a vertex v is called the **degree of that vertex** and is denoted by **deg(v)**.



Vertex	deg(v)
a	2
b	4
c	2
d	4
e	5

Basic Terminologies

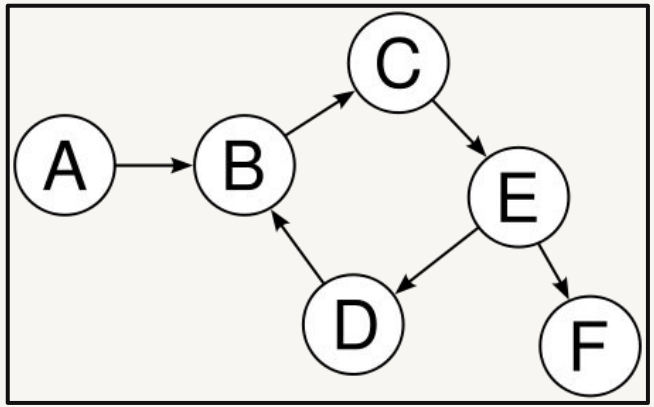
- **In Degree:** The **number of incoming edges** to a vertex v is called the **in-degree of the vertex** and is denoted by **indeg(v)**.
- **Out Degree:** The **number of outgoing edges** from a vertex v is called the **out-degree** and is denoted by **outdeg(v).**



Vertex	indeg(v)	outdeg(v)
a	0	2
b	2	2
c	2	0
d	2	2
e	2	3

Basic Terminologies

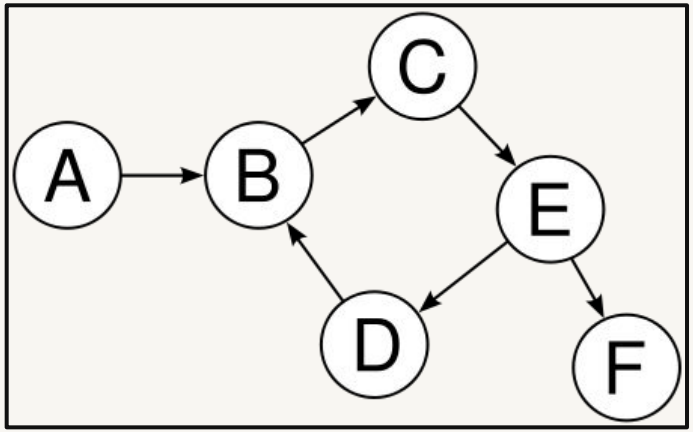
- **Source Vertex: A vertex with in-degree zero**. Generally, in a graph, at least one source vertex is present.



In the graph, vertex A is a source vertex.

Basic Terminologies

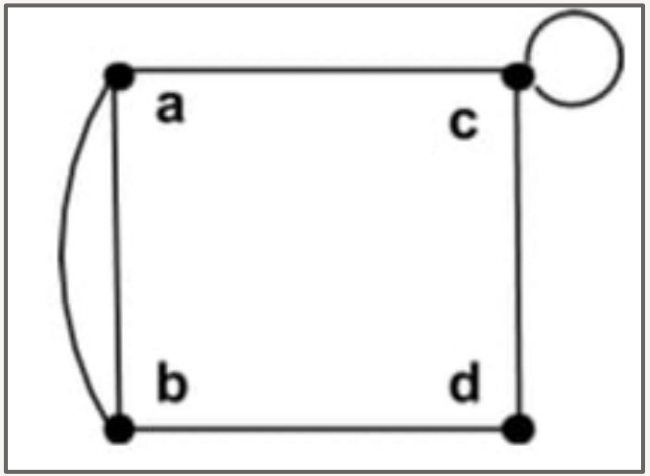
- **Sink Vertex: A vertex with out-degree zero.**



In the graph, vertex F is a sink vertex.

Basic Terminologies

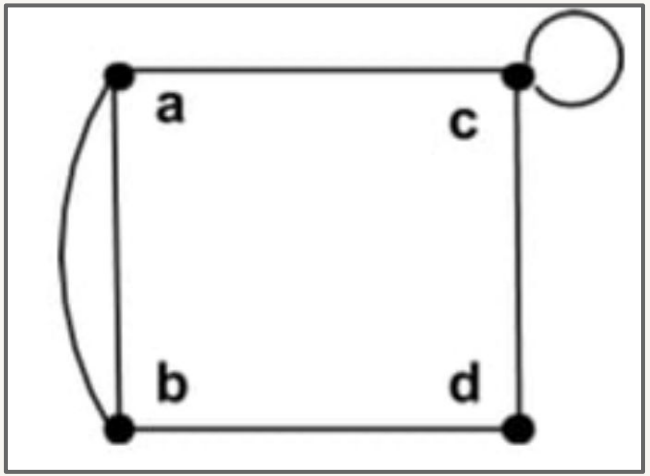
- **Parallel Edges:** Multiple edges between same pair of vertices.



In the graph, vertex a and b has two edges between each other i.e. they are parallel.

Basic Terminologies

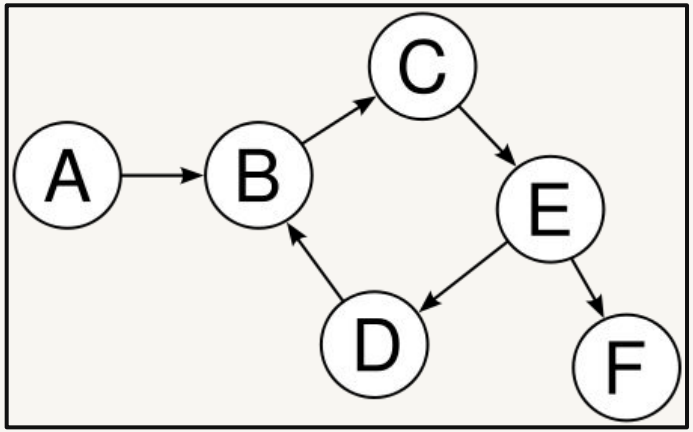
- **Self Loop:** An edge between the vertex and itself.



In the graph, vertex c has an edge which starts from c and ends at c i.e. a self loop.

Basic Terminologies

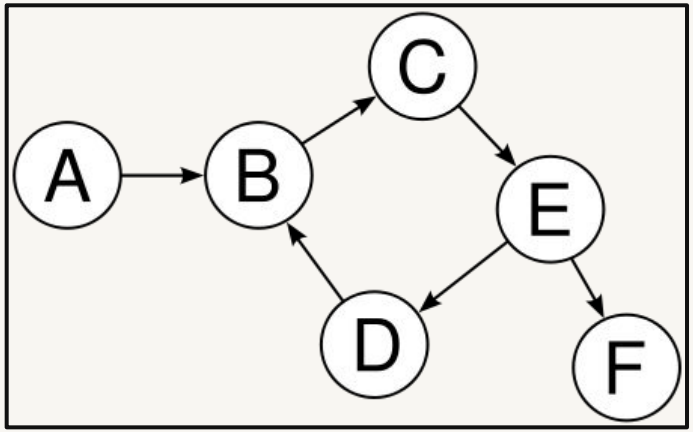
- **Path:** A sequence of vertices connected by edge(s).



In the graph, path from A to D is (A -> B -> C -> E -> D).

Basic Terminologies

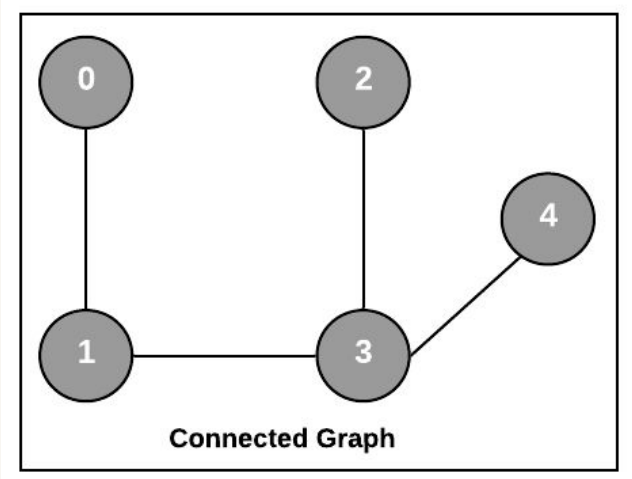
- **Cycle:** A closed path where first and last vertices are the same.



In the graph, a path exists from B to B (B -> C -> E -> D -> B) which is a cycle.

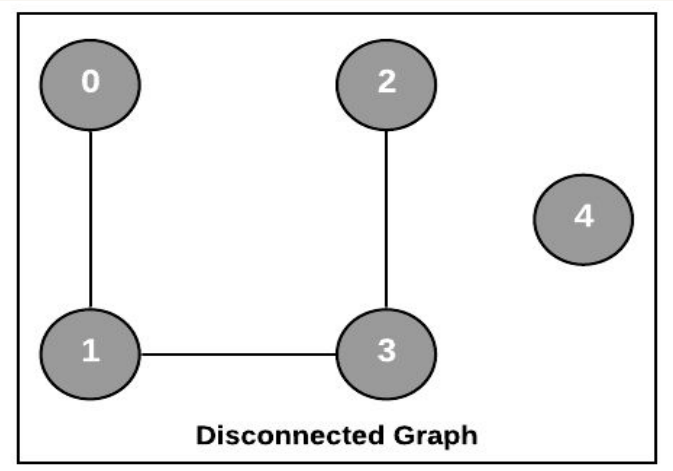
Basic Terminologies

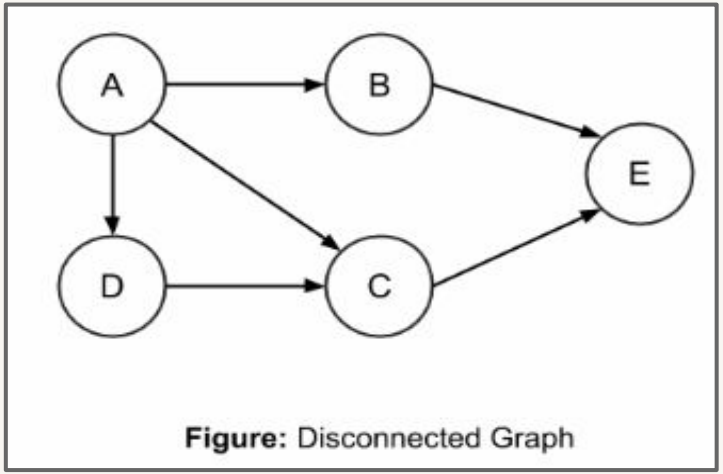
- **Connected Graph:** A path exists between every pair of vertices.



Basic Terminologies

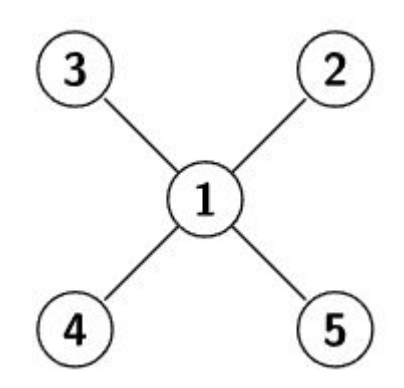
- **Disconnected Graph:** A path is absent between any pair of vertices.





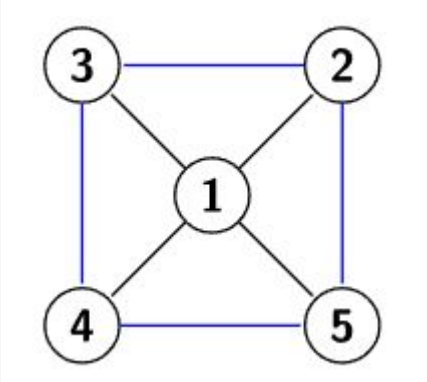
Basic Terminologies

- **Sparse Graph:** Number of edges is considerably less than maximum number of edges.



Basic Terminologies

- **Dense Graph:** Number of edges is close to the maximum number of edges.

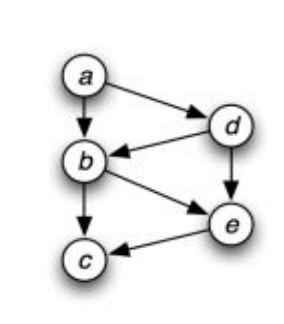


Relationship between n and m

- If G is a graph with m edges, then $\Sigma deg(v) = 2m$
- If G is a directed graph with m edges, then $\Sigma indeg(v) = \Sigma outdeg(v) = m$
- If G is a simple undirected graph with n vertices and m edges, then $m \leq n(n - 1)/2$.
- If G is a simple directed graph with n vertices and m edges, then $m \leq n(n - 1)$.

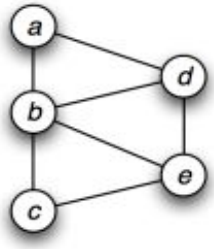
Representation of Graphs

- **Adjacency Matrix:**
  - A graph G = (V, E) can be represented in an matrix of size n x n.
  - If $(v_i, v_j) \in E$, then $a_{i,j} = 1$ and 0 otherwise. In case of undirected graph, $a_{i,j} = 1$ and also $a_{j,i} = 1$.
  - Memory complexity is **$O(n^2)$**



	a	b	c	d	e
a	0	1	0	1	0
b	0	0	1	0	1
c	0	0	0	0	0
d	0	1	0	0	1
e	0	0	1	0	0

Directed Graph

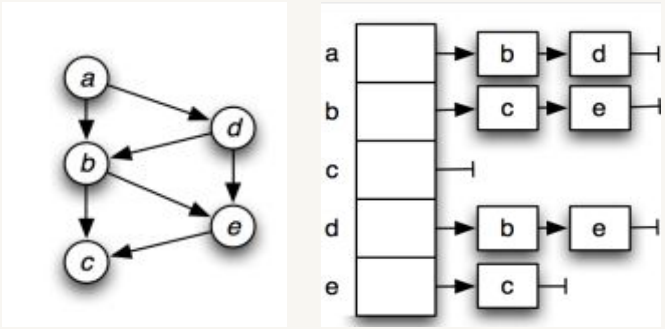


	a	b	c	d	e
a	0	1	0	1	0
b	1	0	1	1	1
c	0	1	0	0	1
d	1	1	0	0	1
e	0	1	1	1	0

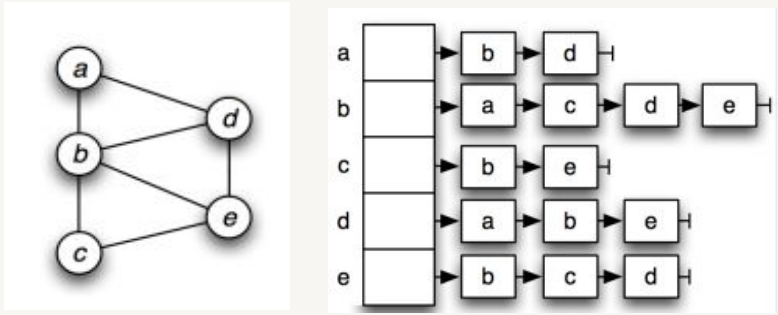
Undirected Graph

Representation of Graphs

- **Adjacency List:**
  - Represents the graph by listing for each vertex v_i, it's outgoing/connected vertices in a list ***out(v_i)***. (Can be represented using a Linked List or any other appropriate data structure)
  - Memory complexity is **O(m + n)**



Directed Graph



Undirected Graph

Graph Traversal

- Given a graph G = (V, E), and a source vertex s, we want to visit all the vertices $\in$ V exactly once!
- Two graph traversal algorithms: Breadth First Search (BFS) and Depth First Search (DFS)

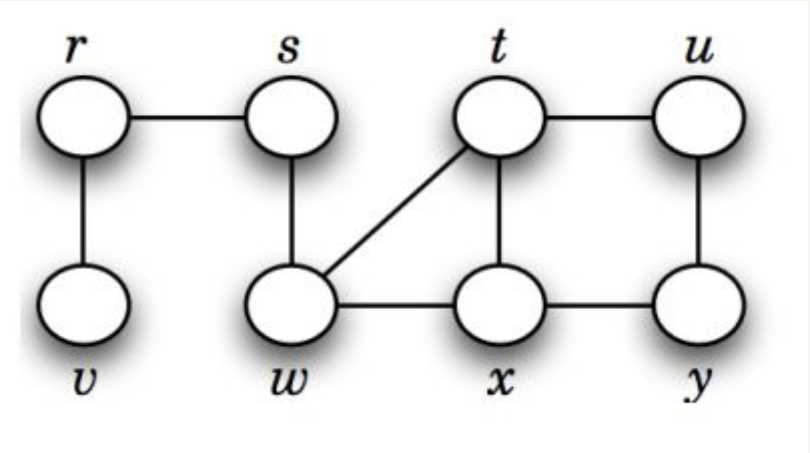
Breadth First Search (BFS)

- Traverses the entire graph level-wise.
- Uses **Queue** for traversal.

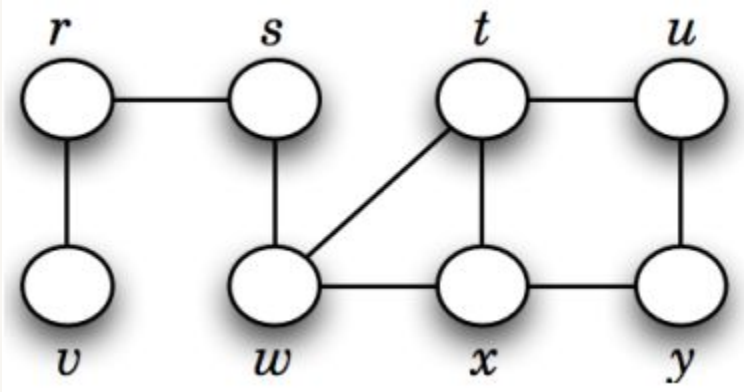
If we start from source vertex s,

then first level vertices are **r, w**
second level vertices are **v, t, x**
third level vertices are **u, y**

So, **BFS traversal** order will be,
s, r, w, v, t, x, u, y



Simulation



Visited

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

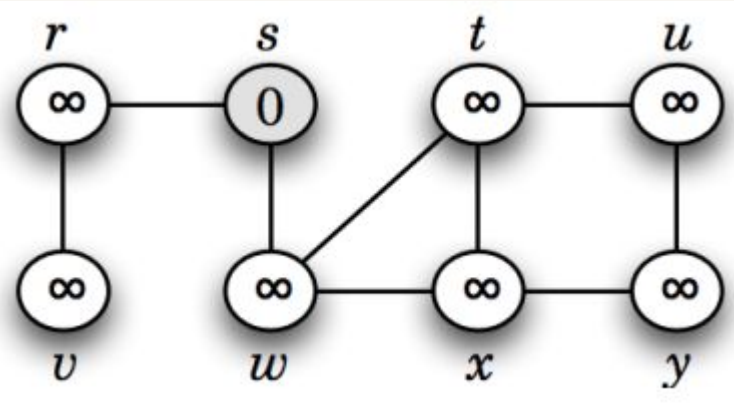
Distance

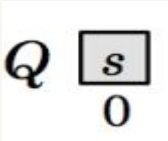
0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Parent

N	N	N	N	N	N	N	N
r	s	t	u	v	w	x	y

Simulation





Visited

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

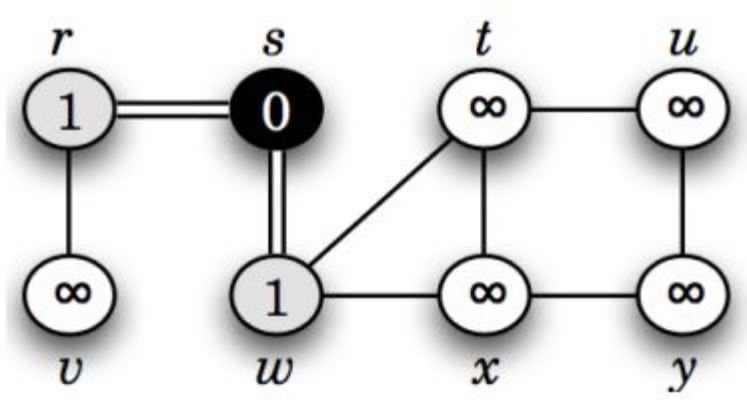
Distance

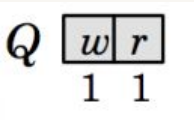
0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Parent

N	-1	N	N	N	N	N	N
r	s	t	u	v	w	x	y

Simulation





Visited

1	1	0	0	0	1	0	0
r	s	t	u	v	w	x	y

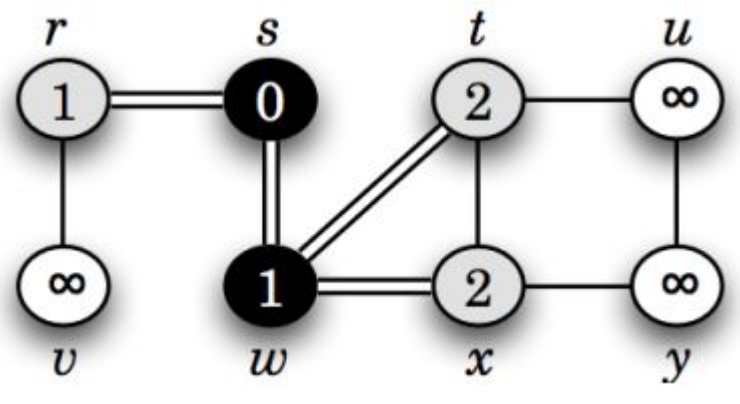
Distance

1	0	0	0	0	1	0	0
r	s	t	u	v	w	x	y

Parent

s	-1	N	N	N	s	N	N
r	s	t	u	v	w	x	y

Simulation



Q	r	t	x
	1	2	2

Visited

1	1	1	0	0	1	1	0
r	s	t	u	v	w	x	y

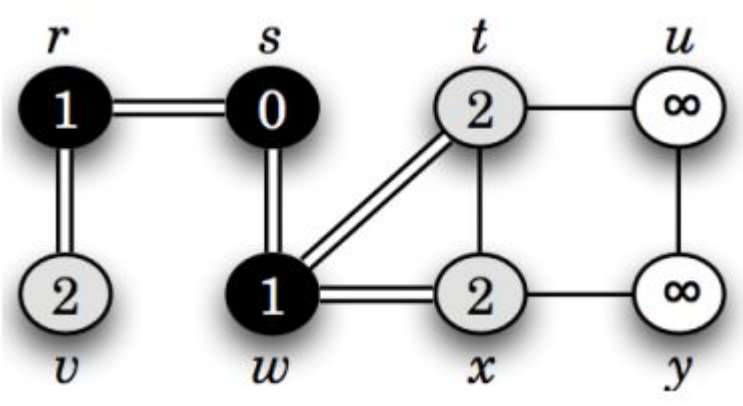
Distance

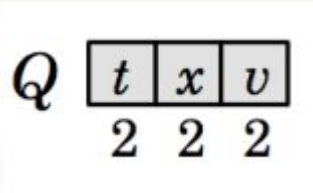
1	0	2	0	0	1	2	0
r	s	t	u	v	w	x	y

Parent

s	-1	w	N	N	s	w	N
r	s	t	u	v	w	x	y

Simulation





Visited

1	1	1	0	1	1	1	0
r	s	t	u	v	w	x	y

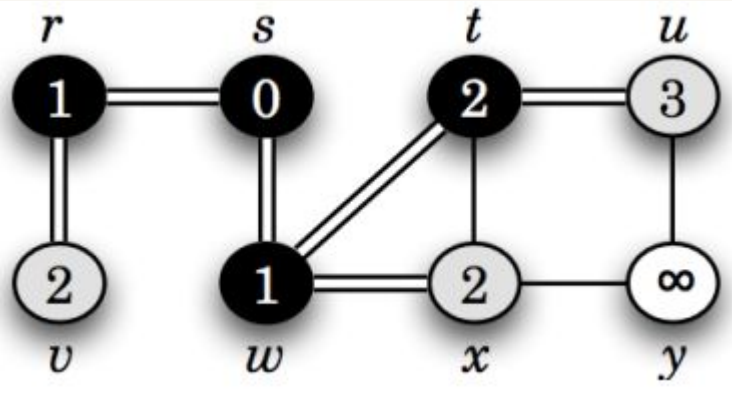
Distance

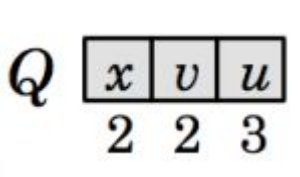
1	0	2	0	2	1	2	0
r	s	t	u	v	w	x	y

Parent

s	-1	w	N	r	s	w	N
r	s	t	u	v	w	x	y

Simulation





Visited

1	1	1	1	1	1	1	0
r	s	t	u	v	w	x	y

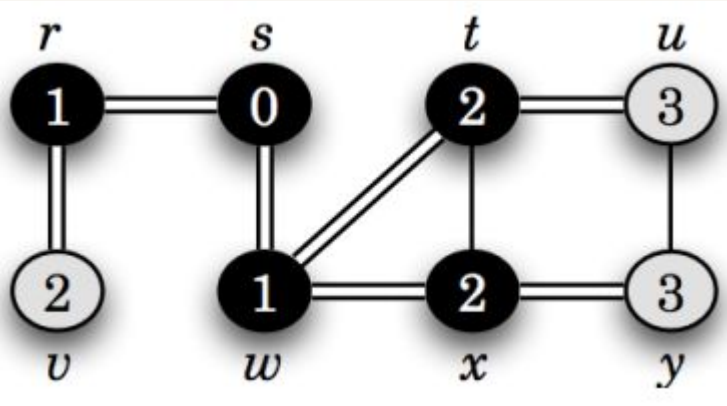
Distance

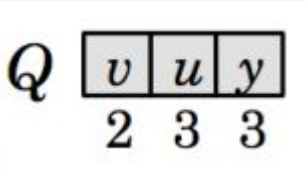
1	0	2	3	2	1	2	0
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	N
r	s	t	u	v	w	x	y

Simulation





Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

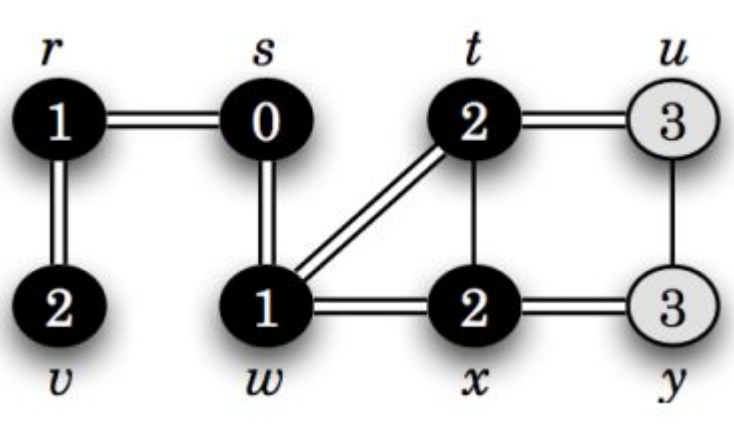
Distance

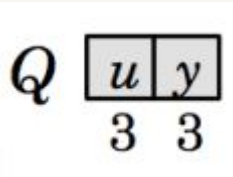
1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

Simulation





Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

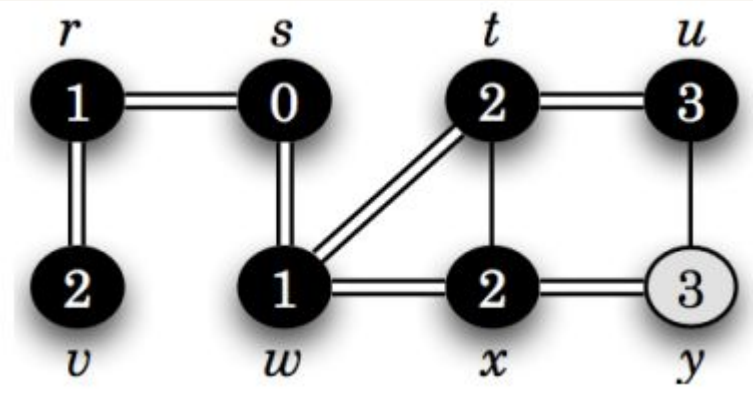
Distance

1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

Simulation



Q $\boxed{y}$
3

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

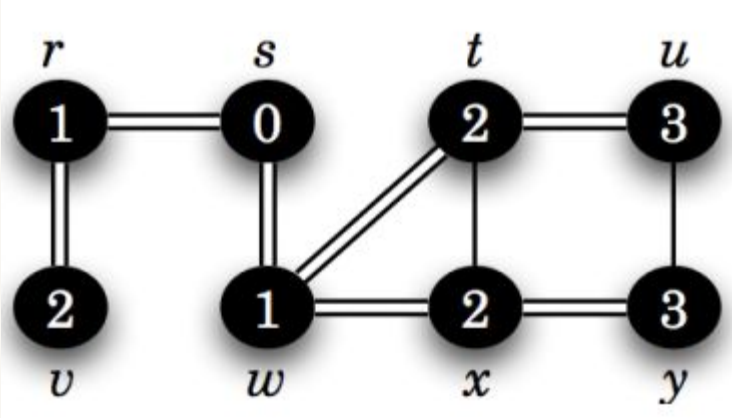
Distance

1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

Simulation



Q $\varnothing$

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

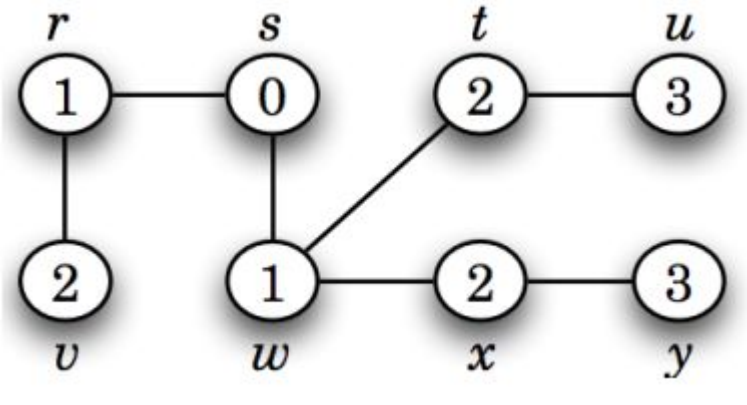
Distance

1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

Simulation



BFS traversal sequence will be,

s, r, w, v, t, x, u, y

Simulation

Visited

1	1	1	1	1	1	1	1
r	**s**	**t**	**u**	**v**	**w**	**x**	**y**

Distance

1	0	2	3	2	1	2	3
r	**s**	**t**	**u**	**v**	**w**	**x**	**y**

Parent

s	-1	w	t	r	s	w	x
r	**s**	**t**	**u**	**v**	**w**	**x**	**y**

What is the shortest distance between s to u?

Ans: Go to index u of distance array (3)

What is the shortest path from s to u?

Ans: Go to index u of parent array (t). Since t is not source, go to index t (w). w is not the source, so go to index w (s). Since s is source, path will be **s -> w -> t -> u**.

BFS Pseudocode

```
BFS(G, s)
Q = [], visited = [], parent = [], distance = []
visited[s] = 1, parent[s] = -1, distance[s] = 0
ENQUEUE(Q, s)
while Q is not empty
    u = DEQUEUE(Q) //Dequeued sequence is the BFS sequence
    for each vertex v Є Adj[u]
        if visited[v] = 0
            visited[v] = 1
            distance[v] = distance[u] + 1
            parent[v] = u
            ENQUEUE(Q, v)
```

BFS Properties

- Can find the shortest path and minimum distance required to traverse from a source vertex to all other vertices in an unweighted graph.
- Can always find the solution.
- In case of directed graph, not all vertices may be explored.
- Not suitable for detecting cycles in a graph.

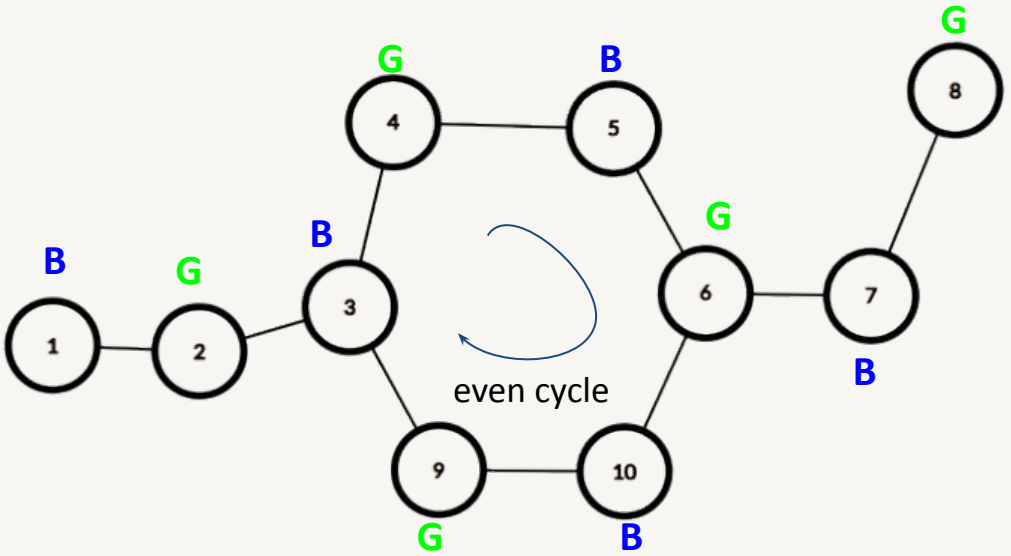
Time Complexity: O(n + m)
Space Complexity: O(n + m)

Bipartite/Bicolorable Graph

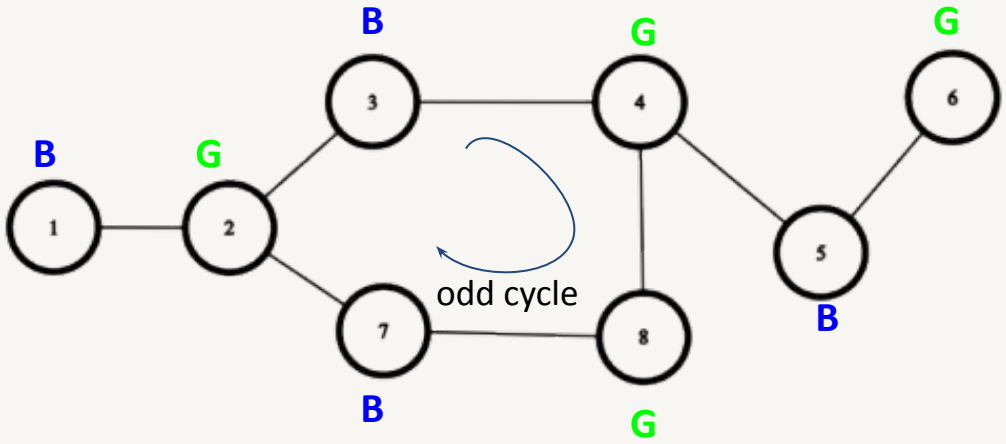
- A graph that can be colored using exactly two colors
- However, no two adjacent vertices should have the same color

Let’s assume two colors,

B -> Blue
G -> Green



Bipartite/Bicolorable Graph



This is not bipartite graph!

Bipartite Graph Detection

- Therefore, it can be said that, any graph that has
  - Odd Cycle -> is not a bipartite graph
  - Even Cycle -> is a bipartite graph

- Similarly,
  - Bipartite Graph -> means it has even cycle
  - Non Bipartite Graph -> means it has odd cycle

Bipartite Graph Detection using BFS

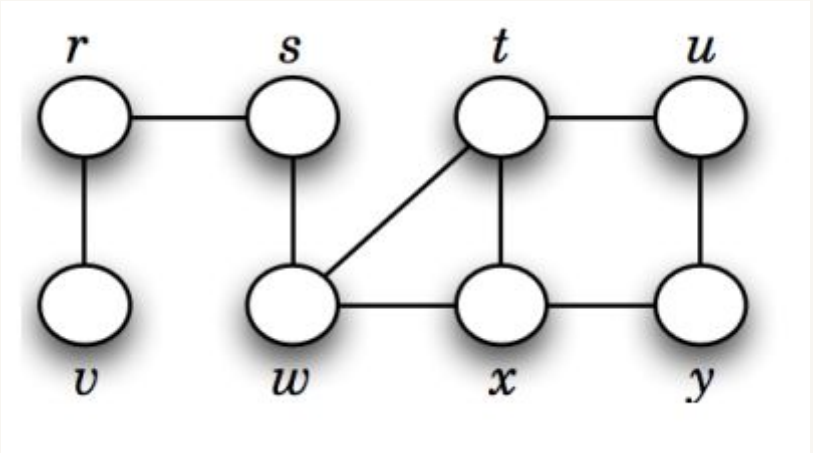
```
Bipartite(G, s)
Q = [], visited = [], color = []
visited[s] = 1, color[s] = "B"
ENQUEUE(Q, s)
while Q is not empty
    u = DEQUEUE(Q)
    for each vertex v Є Adj[u]
        if visited[v] = 0 then visited[v] = 1
            if color[u] == "B" then color[v] = "G"
            else color[v] = "B"
            ENQUEUE(Q, v)
        else if color[u] == color[v] return False
return True
```

Depth First Search (DFS)

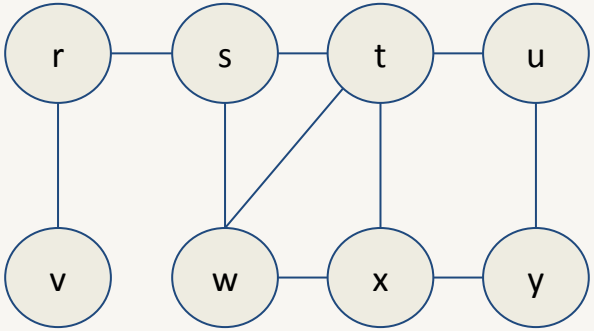
- Traverses the entire graph depth wise without regard for which edge it takes next until it can not go any further at which point it backtracks and continues.
- Uses Stack for traversal.

If we start from source vertex s,
we can go to w, then to x, then to y, then u, then t. After that, we can backtrack from t to u, u to y, y to x, x to w and then to s. From s we can go deeper to r and then to v.

So, one DFS sequence will be,
s, w, x, y, u, t, r, v



Simulation



Stack

Visited

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

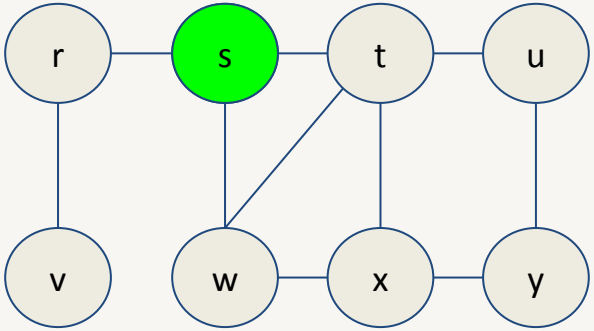
Start Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

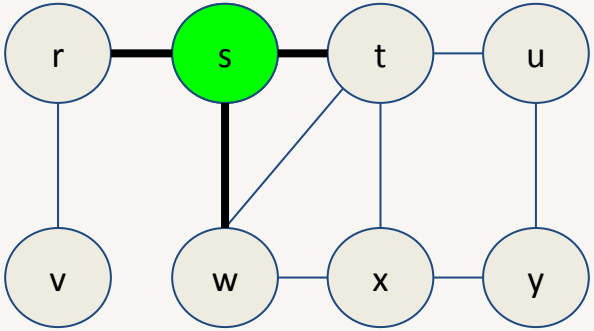
Start Time

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

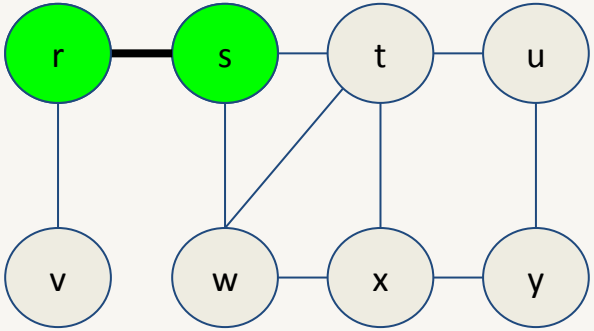
Start Time

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r

Stack

Visited

1	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

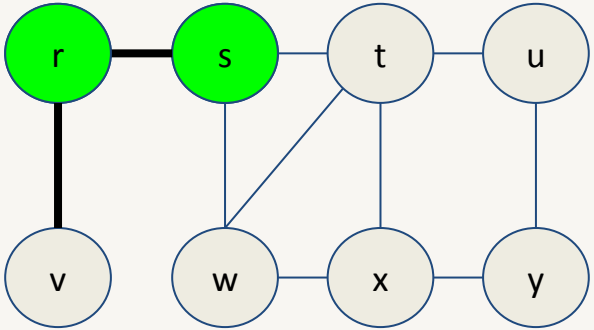
Start Time

2	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r

Stack

Visited

1	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

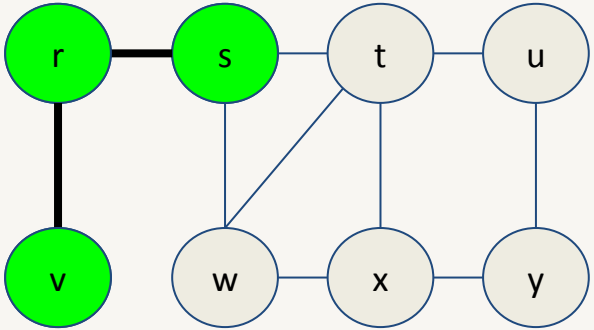
Start Time

2	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r v

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

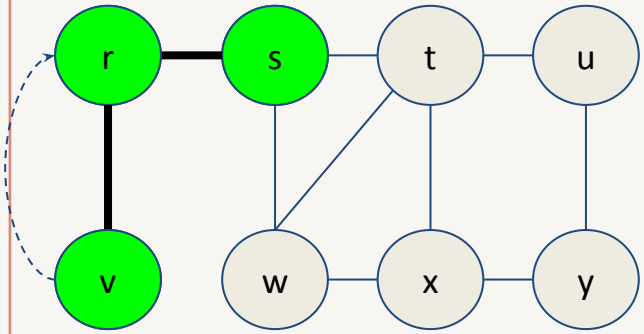
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Simulation



s r

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

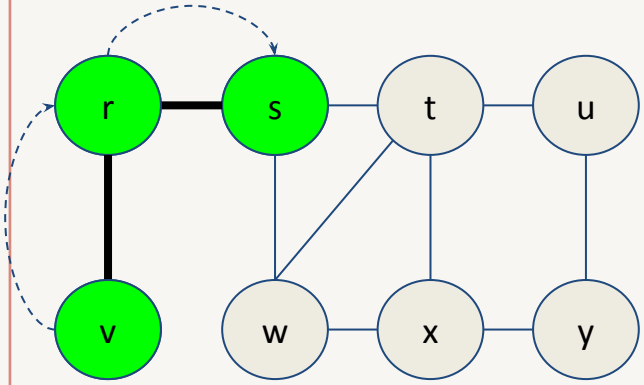
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

0	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

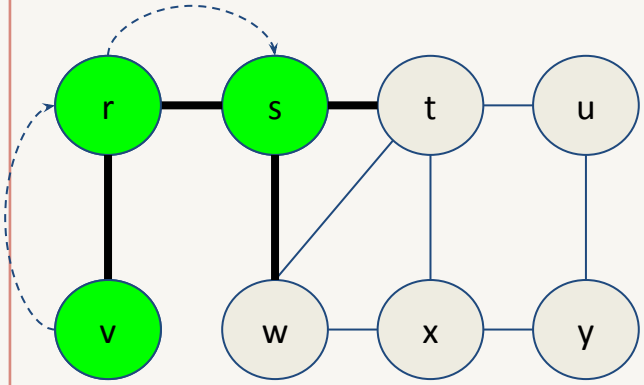
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

1	1	0	0	1	0	0	0
r	s	t	u	v	w	x	y

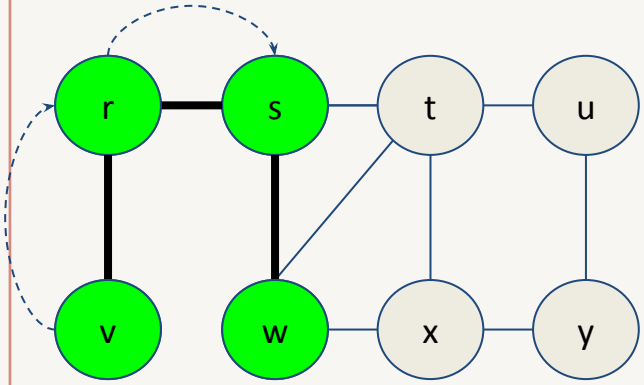
Start Time

2	1	0	0	3	0	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w

Stack

Visited

1	1	0	0	1	1	0	0
r	s	t	u	v	w	x	y

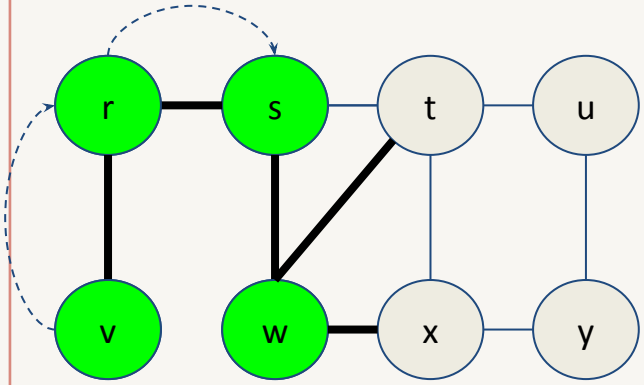
Start Time

2	1	0	0	3	6	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w

Stack

Visited

1	1	0	0	1	1	0	0
r	s	t	u	v	w	x	y

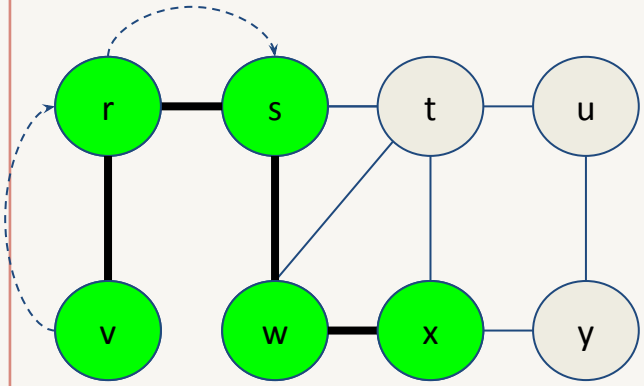
Start Time

2	1	0	0	3	6	0	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x

Stack

Visited

1	1	0	0	1	1	1	0
r	s	t	u	v	w	x	y

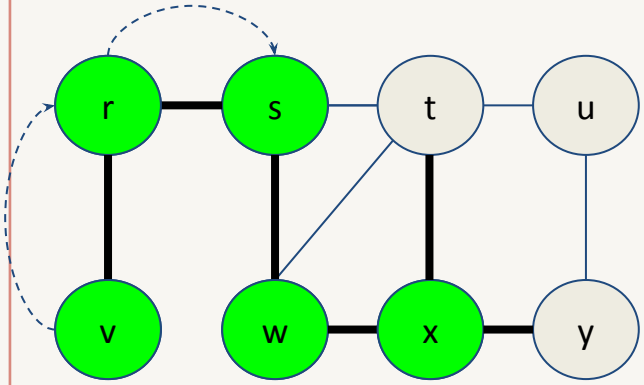
Start Time

2	1	0	0	3	6	7	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x

Stack

Visited

1	1	0	0	1	1	1	0
r	s	t	u	v	w	x	y

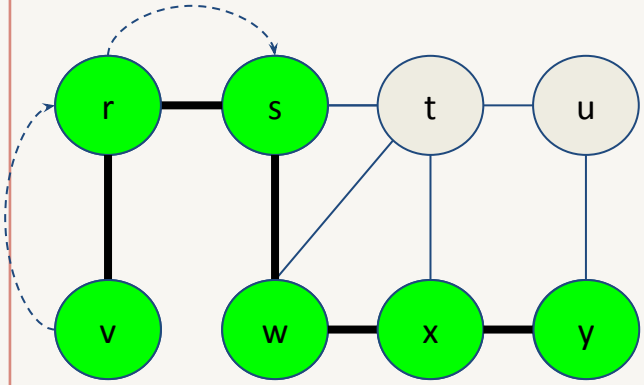
Start Time

2	1	0	0	3	6	7	0
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y

Stack

Visited

1	1	0	0	1	1	1	1
r	s	t	u	v	w	x	y

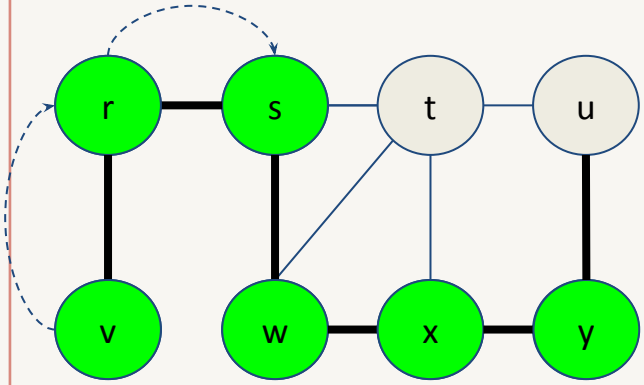
Start Time

2	1	0	0	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x y

Stack

Visited

1	1	0	0	1	1	1	1
r	s	t	u	v	w	x	y

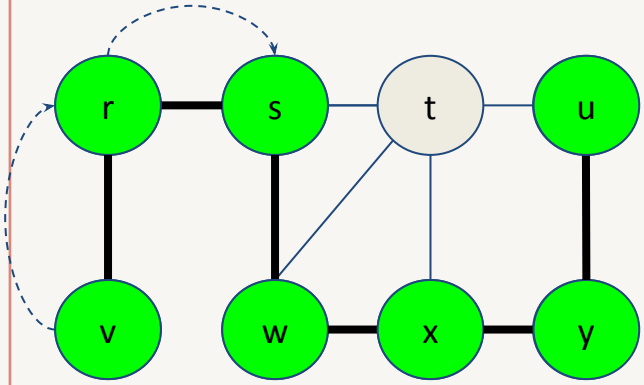
Start Time

2	1	0	0	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u

Stack

Visited

1	1	0	1	1	1	1	1
r	s	t	u	v	w	x	y

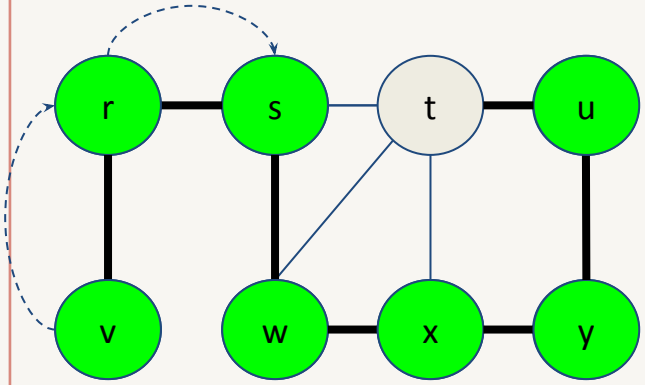
Start Time

2	1	0	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u

Stack

Visited

1	1	0	1	1	1	1	1
r	s	t	u	v	w	x	y

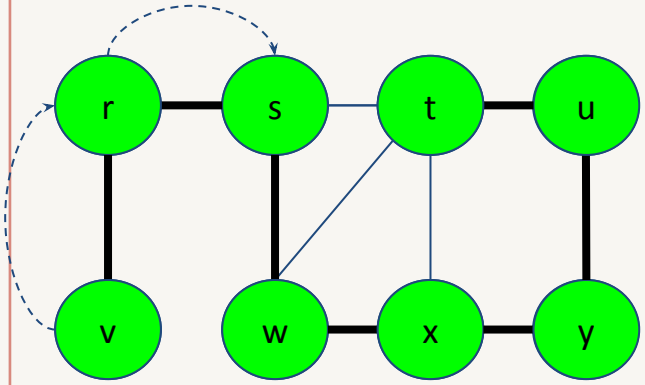
Start Time

2	1	0	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u	t

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

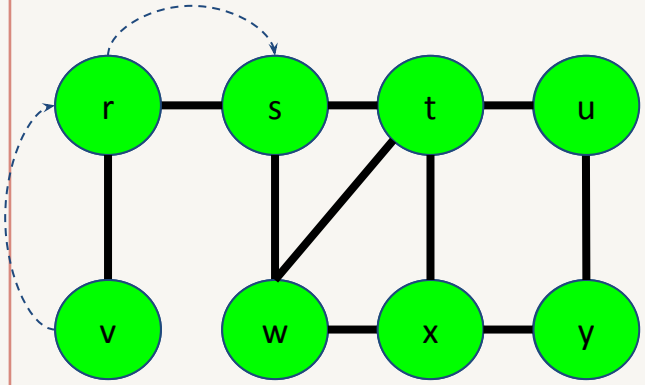
Start Time

2	1	9	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s	w	x	y	u	t

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

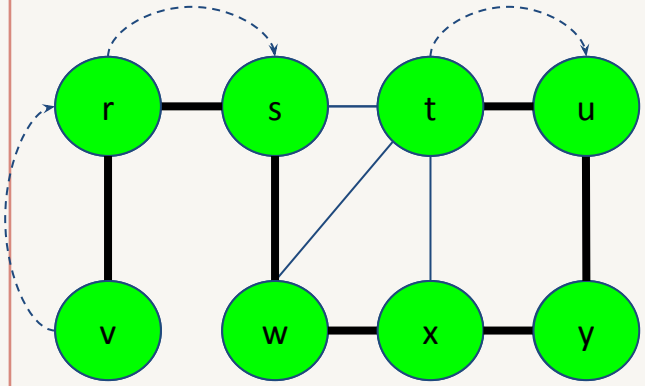
Start Time

2	1	9	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	0	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x y u

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

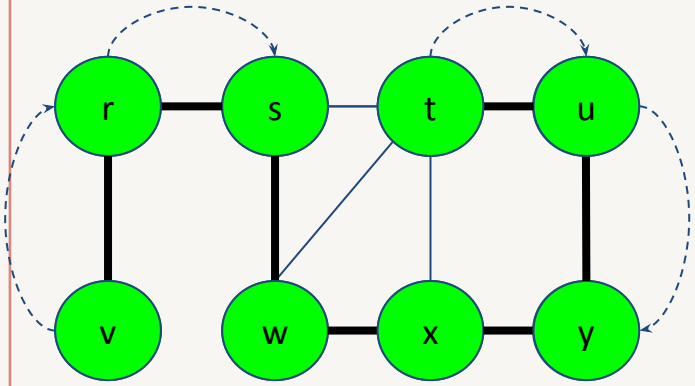
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	0	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x y

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

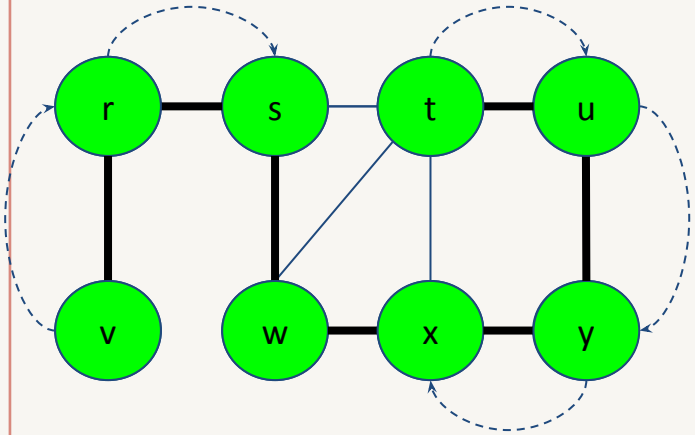
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	0	0	0
r	s	t	u	v	w	x	y

Simulation



s w x

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

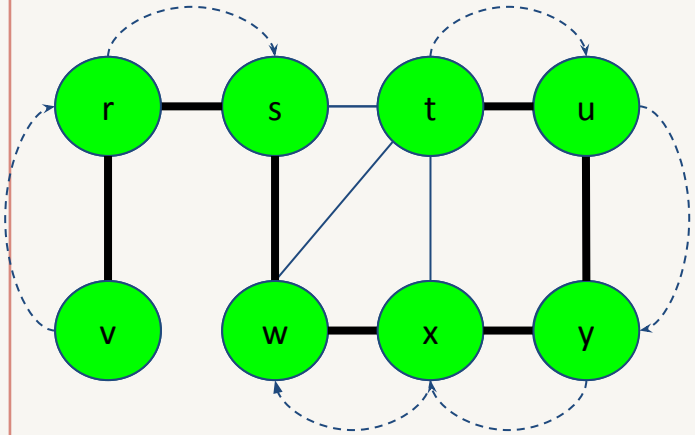
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	0	0	13
r	s	t	u	v	w	x	y

Simulation



s w

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

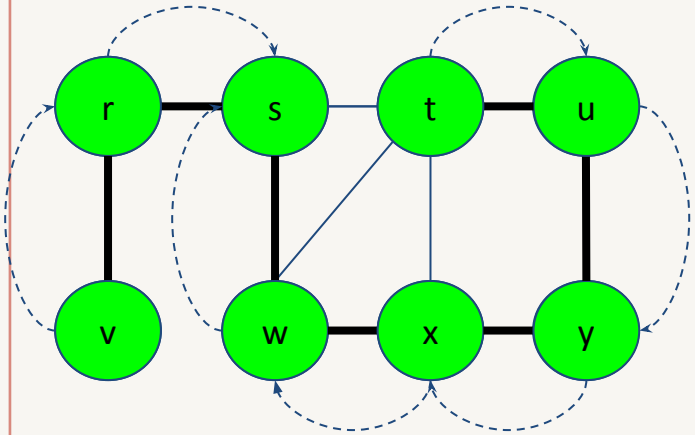
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	0	14	13
r	s	t	u	v	w	x	y

Simulation



s

Stack

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

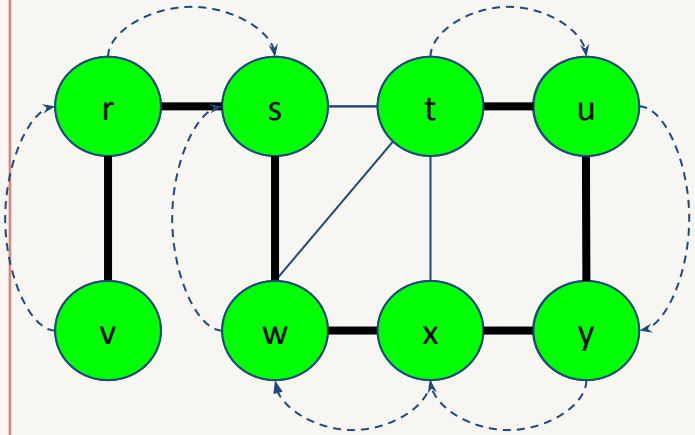
Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

End Time

5	0	11	12	4	15	14	13
r	s	t	u	v	w	x	y

Simulation



Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

Start Time

2	1	10	9	3	6	7	8
r	s	t	u	v	w	x	y

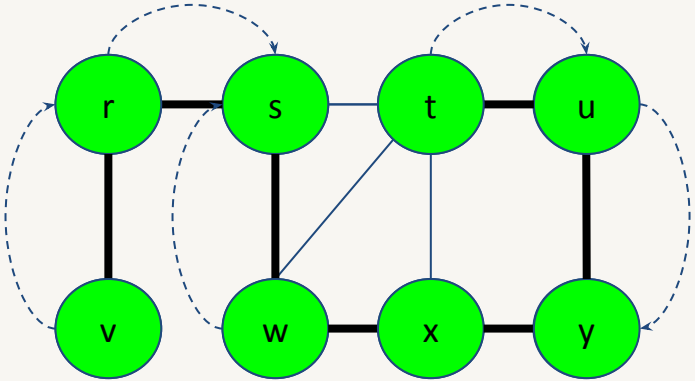
End Time

5	16	11	12	4	15	14	13
r	s	t	u	v	w	x	y

Stack

Since stack is empty, DFS has traversed all vertices!

Simulation



DFS traversal sequence will be,

s, r, v, w, x, y, u, t

DFS Pseudocode

```
visited = [], start = [], end = [], time = 0

DFS(G, s)
    visited[s] = 1
    time = time + 1
    start[s] = time
    for each vertex v ∈ Adj[s]
        if visited[v] = 0
            DFS(G, v)
    time = time + 1
    end[s] = time
```

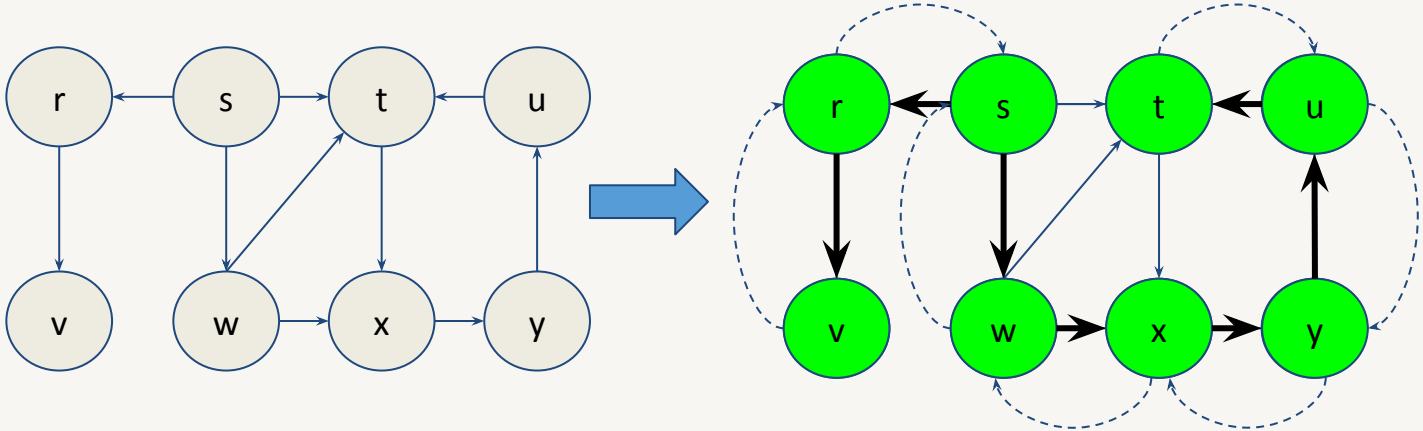
DFS Properties

- Solution is not always guaranteed.
- Can be useful for detection of cycles in a graph.
- In case of directed graph, not all vertices may be explored.

Time Complexity: O(n + m)
Space Complexity: O(n + m)

Edge Classification

- Recall the DFS traversal of this graph

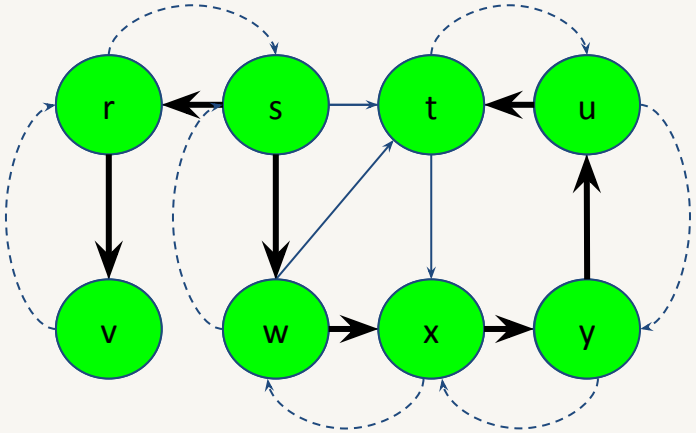


Types of Edge Classification

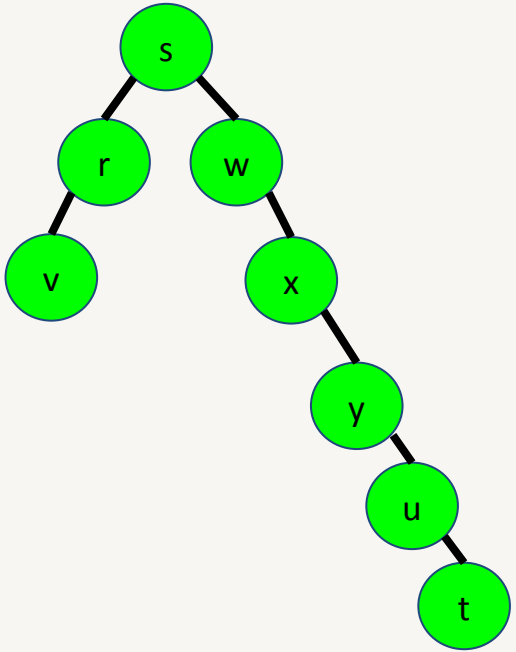
- Based on the DFS Traversal of a particular graph, we can classify the edges of that particular graph into 4 categories

    - Tree edge
    - Forward edge
    - Backward edge
    - Cross edge

Tree Edge



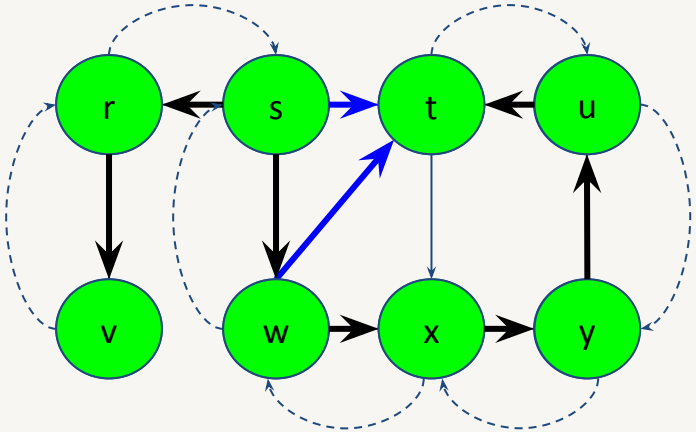




The edges (s, r), (r, v), (s, w), (w, x), (x, y), (y, u) and (u, t) are tree edges as these edges were used for the particular DFS Traversal.

Forward Edge

- An edge (u, v) is forward edge if (u, v) is not a tree edge and **ancestor(v) = u.**



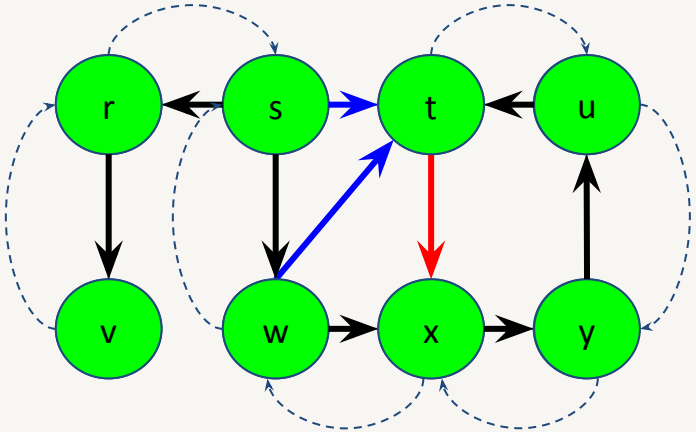
Here, consider the edge (w, t)

(w, t) -> not a tree edge
Ancestor(t) = w (Yes, if you backtrack from t, you will see w in the path). Therefore, (w, t) is a forward edge.

Similarly, (s, t) is also a forward edge!

Backward Edge

- An edge (u, v) is a backward edge if (u, v) is not a tree edge and **ancestor(u) = v.**



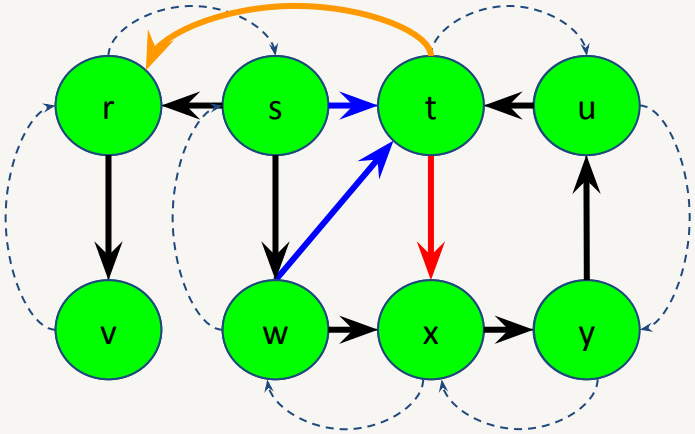
Here, consider the edge (t, x)

(t, x) -> not a tree edge
Ancestor(t) = x (Yes, if you backtrack from t, you will see x in the path). Therefore, (t, x) is a backward edge.

If any graph has any backward edge, that means cycle is present. (t, x, y, u) is a cycle in the graph.

Cross Edge

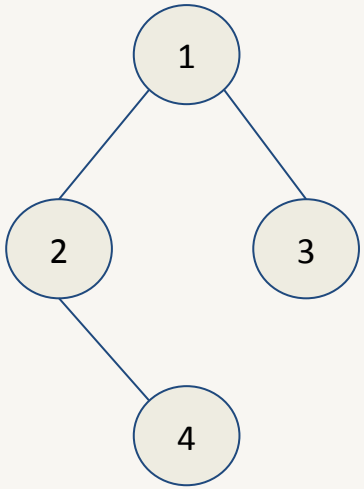
- An edge (u, v) is a cross edge if (u, v) is not a tree edge and ancestor(u) ≠ v or ancestor (v) ≠ u.



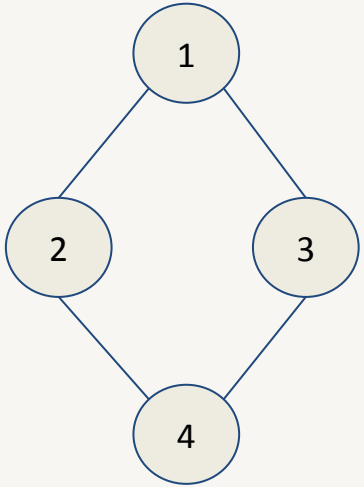
Here, consider if edge (t, r) existed

(t, r) -> not a tree edge
ancestor(t) ≠ r or ancestor(r) ≠ t (Yes, if you backtrack from t to r, you will not find r in the path. Again, if you backtrack from r to t, you will not find t in path).
Therefore, (t, r) is a cross edge.

Cycle in an Undirected Graph

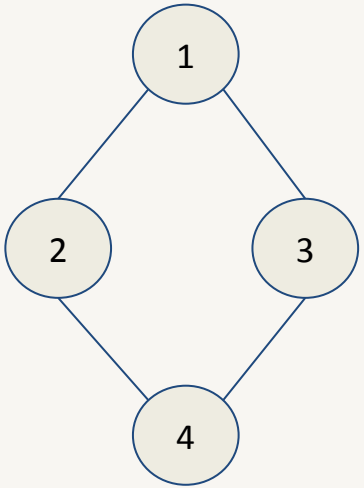


No cycle



Cycle

Cycle Detection using DFS



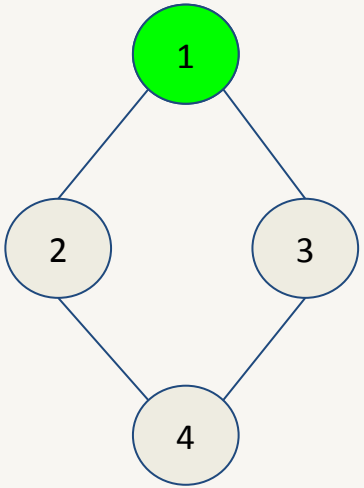
Visited

0	0	0	0
1	2	3	4

Parent

0	0	0	0
1	2	3	4

Cycle Detection using DFS



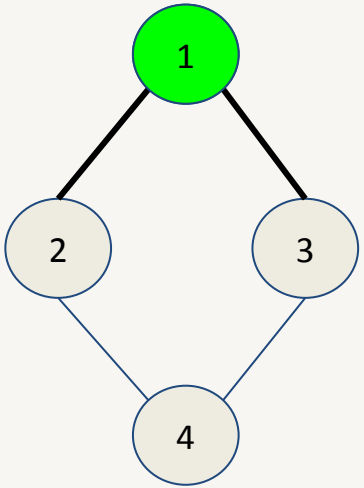
Visited

1	0	0	0
1	2	3	4

Parent

-1	0	0	0
1	2	3	4

Cycle Detection using DFS



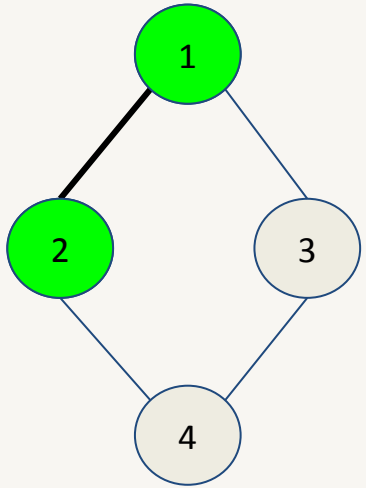
Visited

1	0	0	0
1	2	3	4

Parent

-1	0	0	0
1	2	3	4

Cycle Detection using DFS



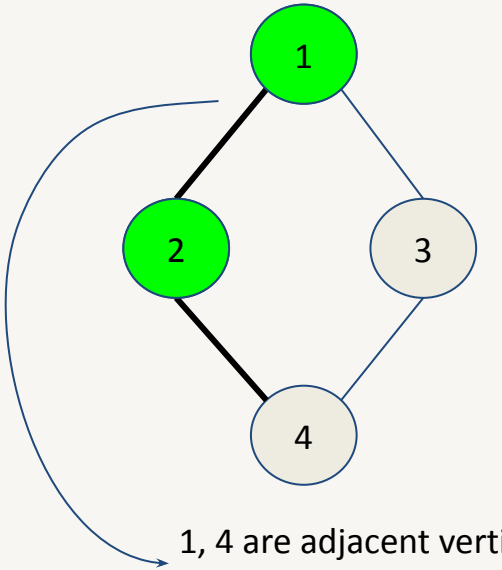
Visited

1	1	0	0
1	2	3	4

Parent

-1	1	0	0
1	2	3	4

Cycle Detection using DFS



Visited

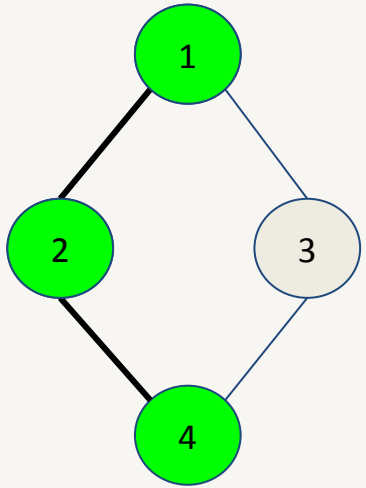
1	1	0	0
1	2	3	4

Parent

-1	1	0	0
1	2	3	4

1, 4 are adjacent vertices of 2. However, 1 is already visited and 1 is also parent of 2. Therefore, it is not a cycle and we will visit 4 since it is unvisited yet.

Cycle Detection using DFS



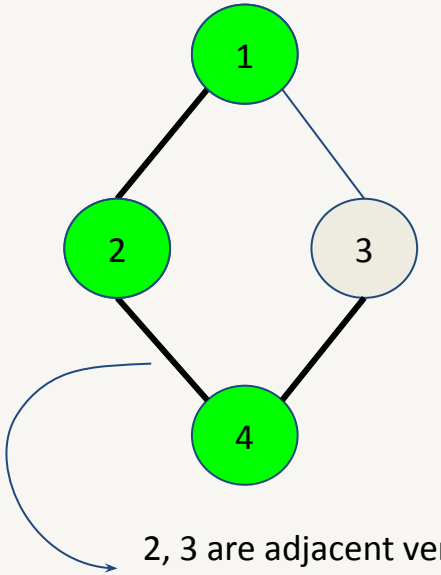
Visited

1	1	0	1
1	2	3	4

Parent

-1	1	0	2
1	2	3	4

Cycle Detection using DFS



Visited

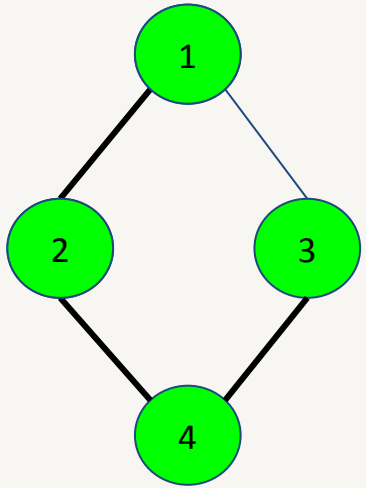
1	1	0	1
1	**2**	**3**	**4**

Parent

-1	1	0	2
1	**2**	**3**	**4**

2, 3 are adjacent vertices of 4. However, 2 is already visited and 2 is also parent of 4. Therefore, it is not a cycle and we will visit 3 as it is unvisited yet.

Cycle Detection using DFS



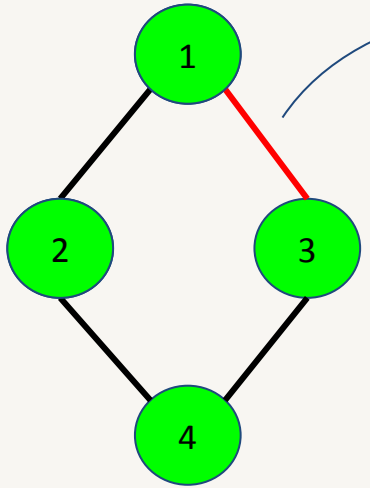
Visited

1	1	1	1
1	**2**	**3**	**4**

Parent

-1	1	4	2
1	**2**	**3**	**4**

Cycle Detection using DFS



Visited

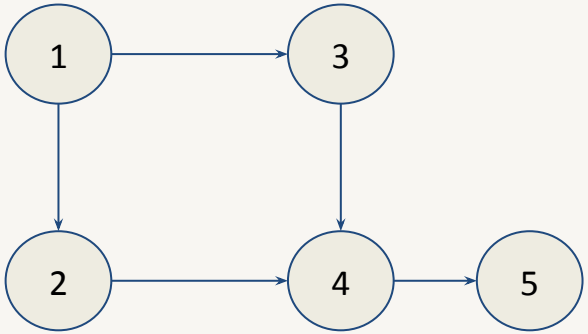
1	1	1	1
1	**2**	**3**	**4**

Parent

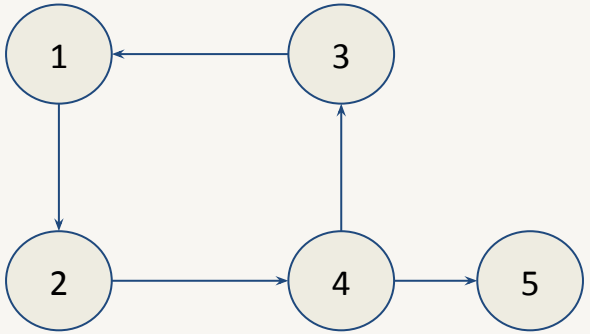
-1	1	4	2
1	**2**	**3**	**4**

4, 1 are adjacent vertices of 3. However, 4 is already visited and 4 is also parent of 3. Therefore, it is not a cycle. We will then check 1 and since 1 is already visited, we will check whether 1 is parent of 3. **Since parent of 3 is 4, therefore, there exists a cycle!**

Cycle in a Directed Graph

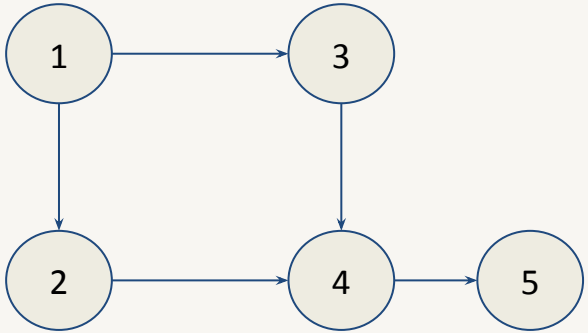


No cycle



Cycle

Cycle Detection using DFS



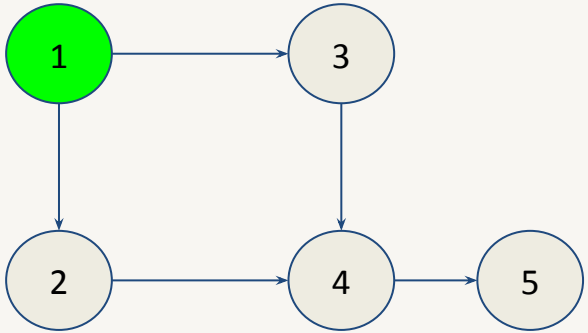
Visited

0	0	0	0	0
1	2	3	4	5

Parent

N	N	N	N	N
1	2	3	4	5

Cycle Detection using DFS



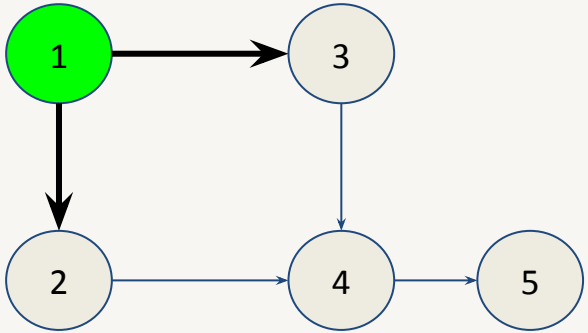
Visited

1	0	0	0	0
1	2	3	4	5

Parent

-1	N	N	N	N
1	2	3	4	5

Cycle Detection using DFS



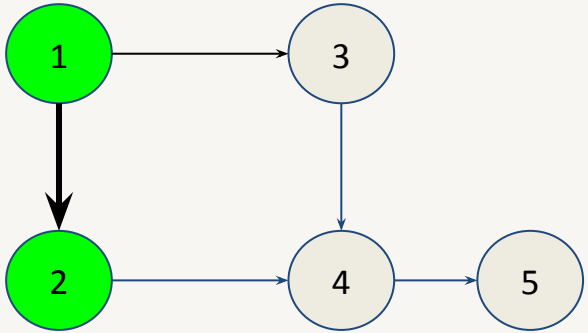
Visited

1	0	0	0	0
1	2	3	4	5

Parent

-1	N	N	N	N
1	2	3	4	5

Cycle Detection using DFS



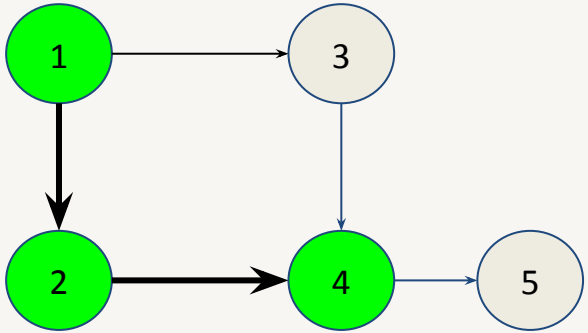
Visited

1	1	0	0	0
1	2	3	4	5

Parent

-1	1	N	N	N
1	2	3	4	5

Cycle Detection using DFS



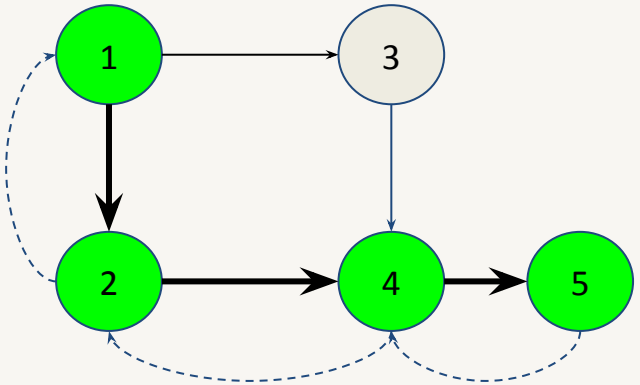
Visited

1	1	0	1	0
1	2	3	4	5

Parent

-1	1	N	2	N
1	2	3	4	5

Cycle Detection using DFS



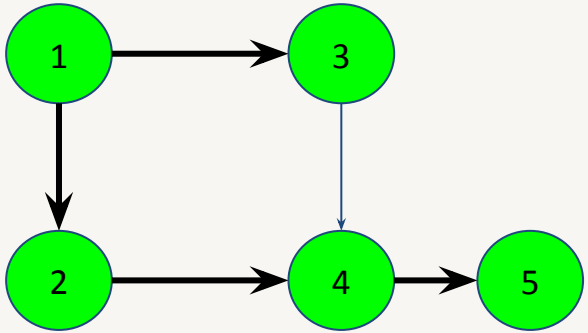
Visited

1	1	0	1	1
1	2	3	4	5

Parent

-1	1	N	2	4
1	2	3	4	5

Cycle Detection using DFS



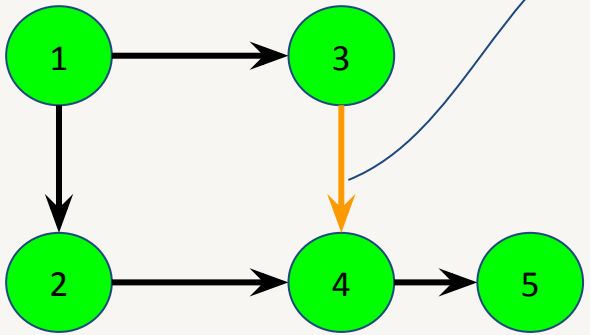
Visited

1	1	1	1	1
1	2	3	4	5

Parent

-1	1	1	2	4
1	2	3	4	5

Cycle Detection using DFS



Visited

1	1	1	1	1
1	2	3	4	5

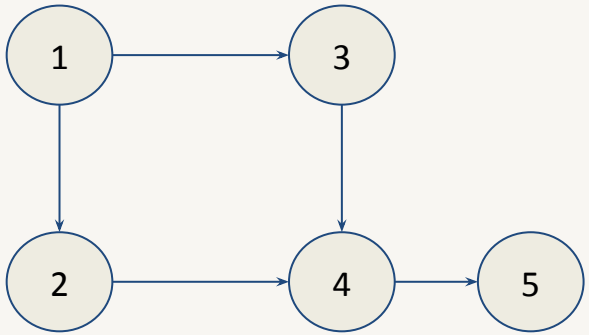
Parent

-1	1	1	2	4
1	2	3	4	5

3 has adjacent vertex 4, which has already been visited. So according to previous way of cycle detection in an undirected graph, we will check parent of 4 which is 2 and not 3. This will return True which means a cycle is present. However, there are no cycles in this graph! Workaround?

Cycle Detection

We will not follow the approach used to detect cycle in an undirected graph. Instead, we will keep a list/stack named currentPath and will check whether the vertex exists in currentPath or not.

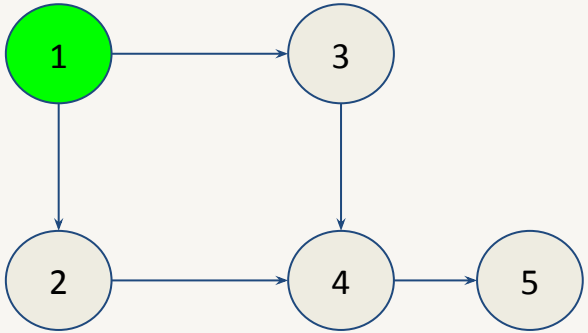


Visited

0	0	0	0	0
1	2	3	4	5

currentPath

Cycle Detection



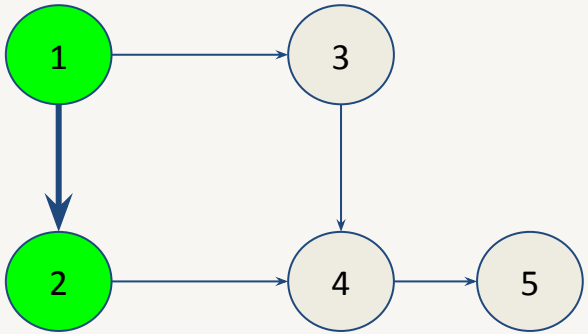
Visited

1	0	0	0	0
1	**2**	**3**	**4**	**5**

1

currentPath

Cycle Detection



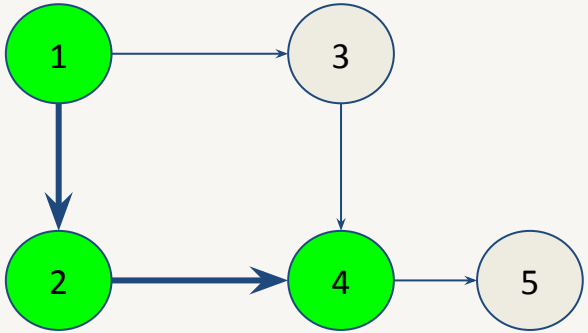
Visited

1	1	0	0	0
1	**2**	**3**	**4**	**5**

1 2

currentPath

Cycle Detection



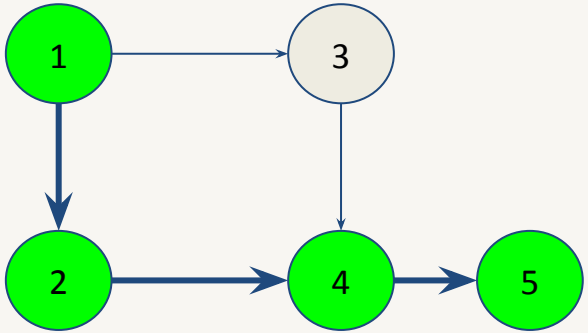
Visited

1	1	0	1	0
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

Cycle Detection



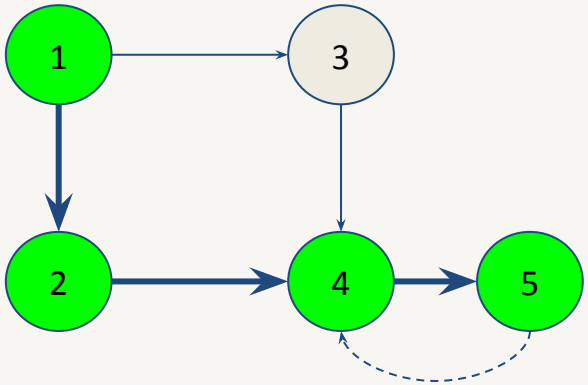
Visited

1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4 5

currentPath

Cycle Detection



Visited

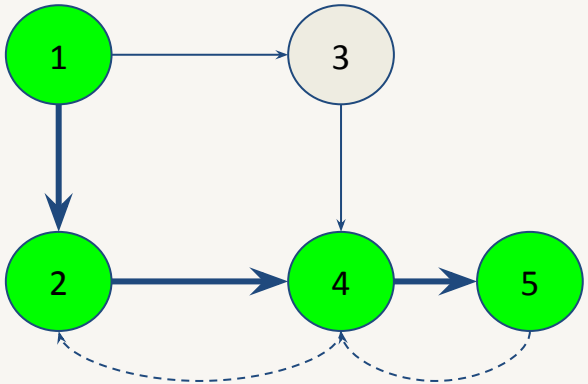
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

As we backtrack from 5 to 4, we will remove 5 from Current Path.

Cycle Detection



Visited

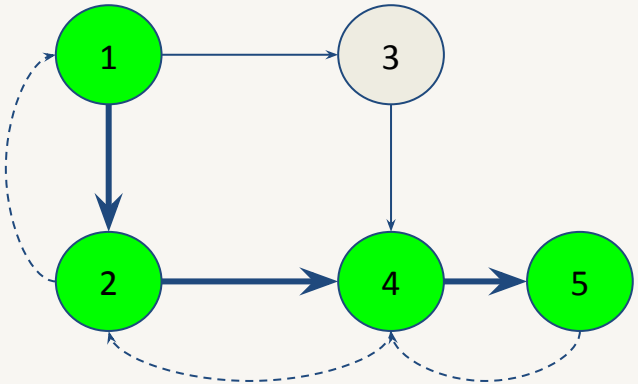
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2

currentPath

As we backtrack from 4 to 2, we will remove 4 from Current Path.

Cycle Detection



Visited

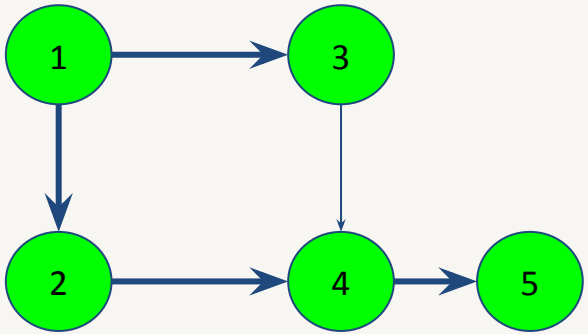
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1

currentPath

As we backtrack from 2 to 1, we will remove 2 from Current Path.

Cycle Detection



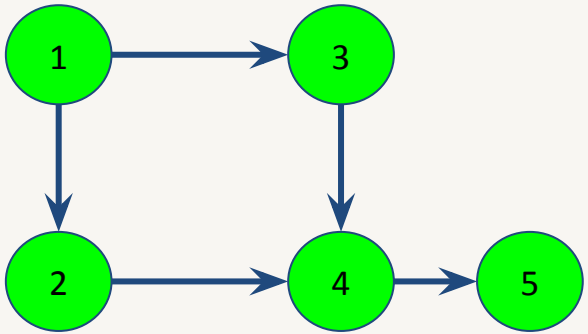
Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

1 3

currentPath

Cycle Detection



Visited

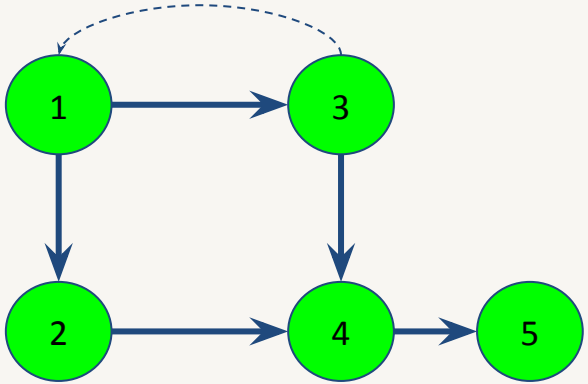
1	1	1	1	1
1	**2**	**3**	**4**	**5**

1 3

currentPath

3 has adjacent vertex 4. However, 4 is already visited. So we will check whether 4 exists in currentPath or not. **Since it does not exist in currentPath, no cycle is present in the above graph.**

Cycle Detection



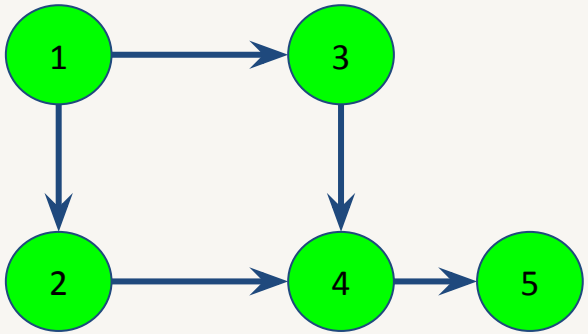
Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

1

currentPath

Cycle Detection

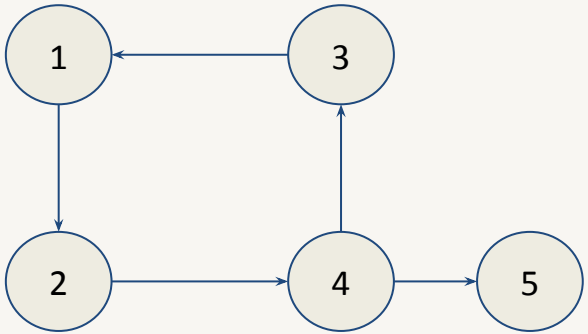


Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

currentPath

Cycle Detection

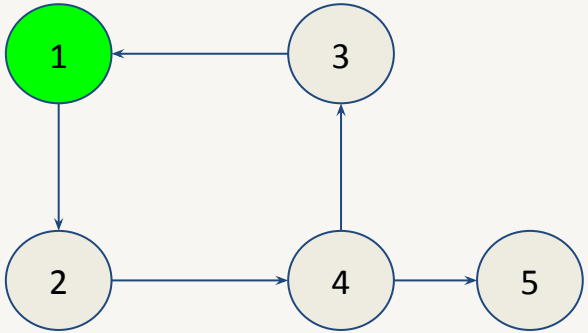


Visited

0	0	0	0	0
1	2	3	4	5

currentPath

Cycle Detection



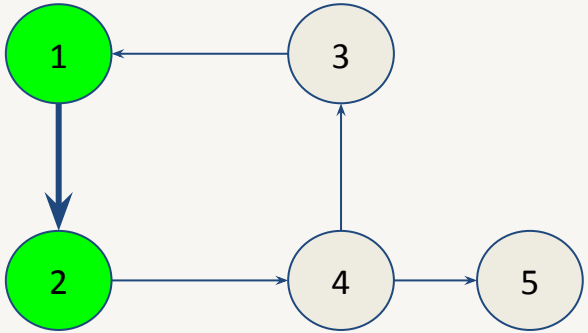
Visited

1	0	0	0	0
1	**2**	**3**	**4**	**5**

1

currentPath

Cycle Detection



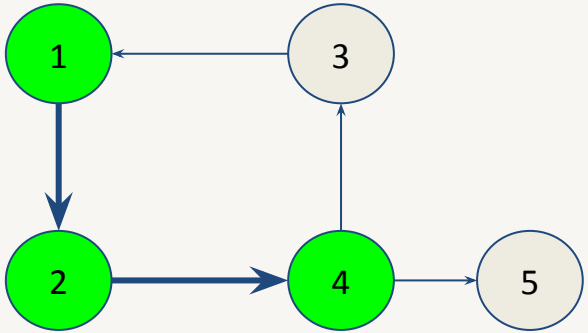
Visited

1	1	0	0	0
1	**2**	**3**	**4**	**5**

1 2

currentPath

Cycle Detection



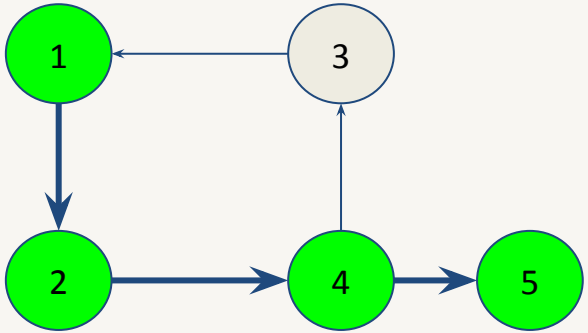
Visited

1	1	0	1	0
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

Cycle Detection



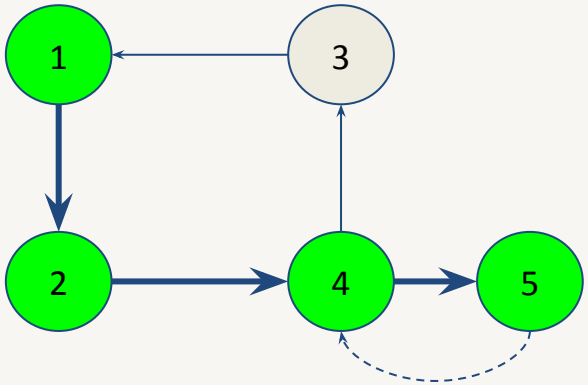
Visited

1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4 5

currentPath

Cycle Detection



Visited

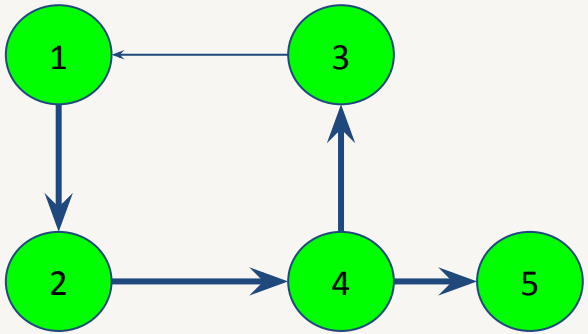
1	1	0	1	1
1	**2**	**3**	**4**	**5**

1 2 4

currentPath

As we backtrack from 5 to 4, we will remove 5 from Current Path.

Cycle Detection



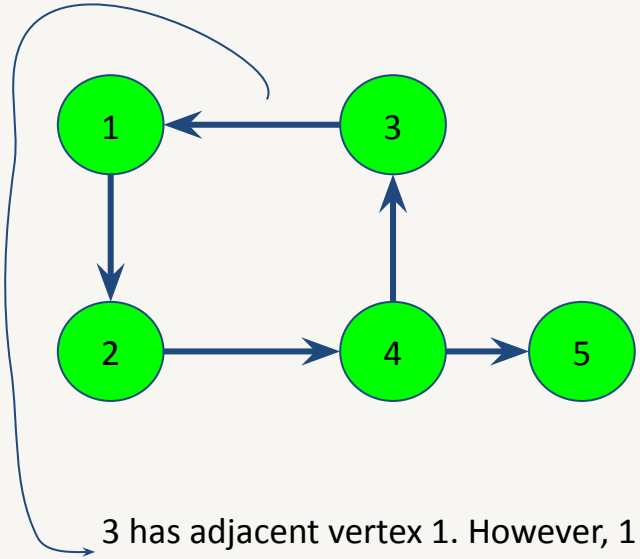
Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

1 2 4 3

currentPath

Cycle Detection



Visited

1	1	1	1	1
1	**2**	**3**	**4**	**5**

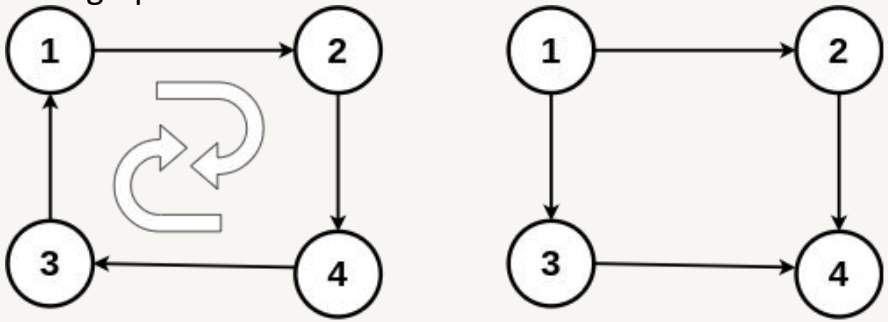
1 2 4 3

currentPath

3 has adjacent vertex 1. However, 1 is already visited. So we will check whether 1 exists in currentPath or not. **Since it exists in currentPath, therefore, cycle is present in the above graph.**

Directed Acyclic Graph (DAG)

- A **Directed Acyclic Graph (DAG)** is a directed graph that does not contain any cycles.
- Directed Acyclic Graph has two important features:
  - **Directed Edges:** In Directed Acyclic Graph, each edge has a direction, meaning it goes from one vertex (node) to another.
  - **Acyclic:** The term "acyclic" indicates that there are no cycles or closed loops within the graph.

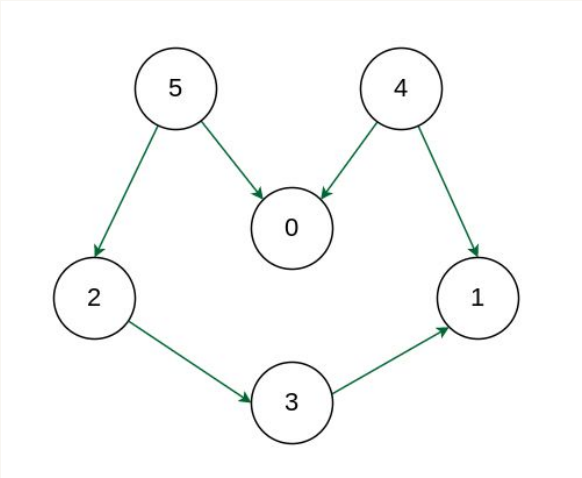


Direct Cyclic Graph

Direct Acyclic Graph

Topological Sorting

- **A linear ordering of vertices such that for every directed edge (u, v), vertex u comes before v in the ordering.**
- Only works for Directed Acyclic Graph (DAG).



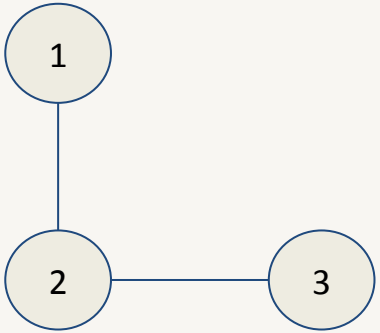
Here, **E = {(2, 3), (3, 1), (4, 0), (4, 1), (5, 0), (5, 2)}**

One possible topological sorting is 4 5 2 3 1 0. Among the edges, one of them is (4, 0). As we can see, 4 comes before 0 in the sequence. Same can be said for (3, 1) and others as well.

There can be more than one topological sorting for a graph.

Topological Sorting

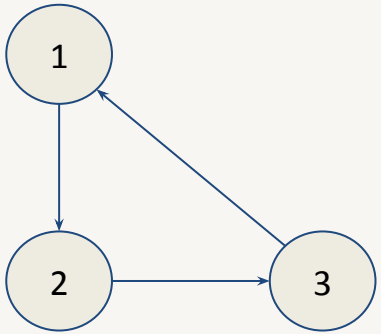
- Why topological sorting is not possible for graphs with undirected edges?
  - This creates a contradiction in the topological ordering as an undirected edge (u, v) means there is an edge from u to v as well as from v to u.



Here, the edge between nodes 1 and 2 means just they are connected, but there's no concept of one coming before the other. As such, contradiction will be created in the ordering.

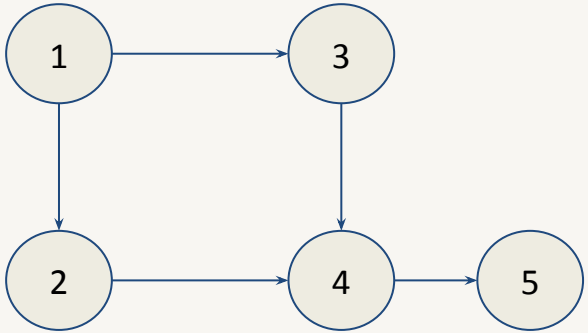
Topological Sorting

- Why topological sorting is not possible for graphs with cycles?
  - This also creates a contradiction in the topological ordering as in a cycle, all the vertices are indirectly dependent on each other.



Here, the vertices (1, 2), (2, 3) and (3, 1) are indirectly dependent on each other. As a result, a topological ordering 1 2 3 contradicts with the edge (3, 1) as here 3 comes before 1.

Topological Sorting using DFS



Visited

0	0	0	0	0
1	2	3	4	5

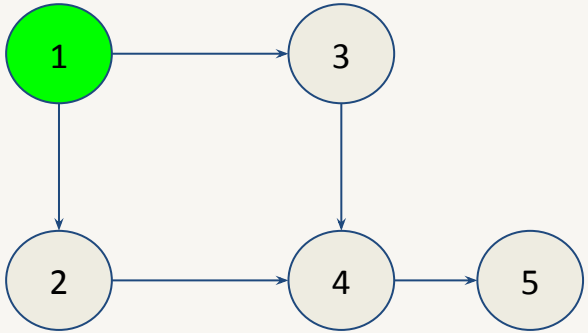
Start Time

0	0	0	0	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	0	0	0
1	2	3	4	5

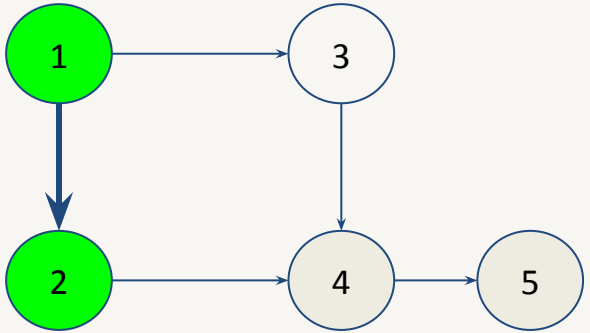
Start Time

1	0	0	0	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	0	0
1	2	3	4	5

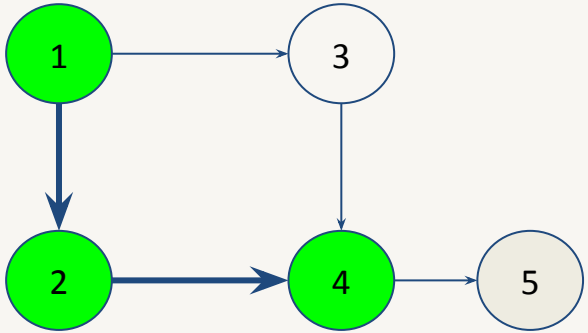
Start Time

1	2	0	0	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	1	0	0
1	2	3	4	5

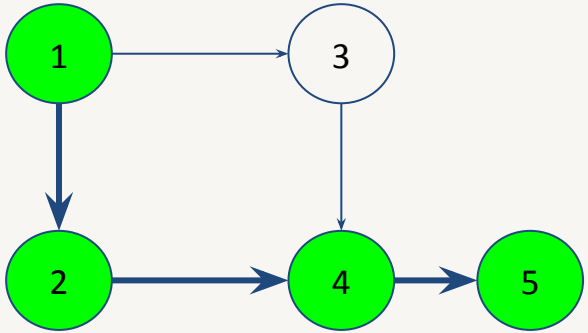
Start Time

1	2	0	3	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	1	1
1	2	3	4	5

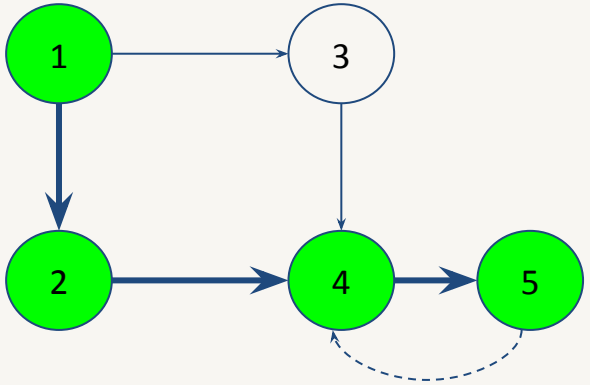
Start Time

1	2	0	3	4
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	1	1	1
1	2	3	4	5

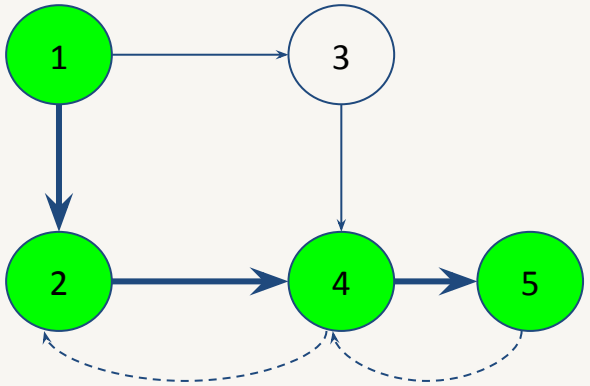
Start Time

1	0	2	3	4
1	2	3	4	5

End Time

0	0	0	0	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	1	1	1
1	2	3	4	5

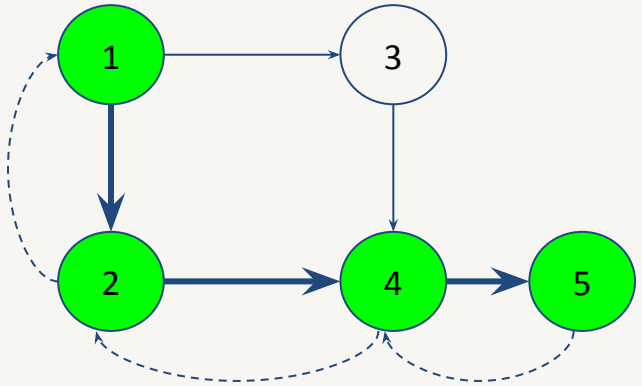
Start Time

1	0	2	3	4
1	2	3	4	5

End Time

0	0	0	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	1	1	1
1	2	3	4	5

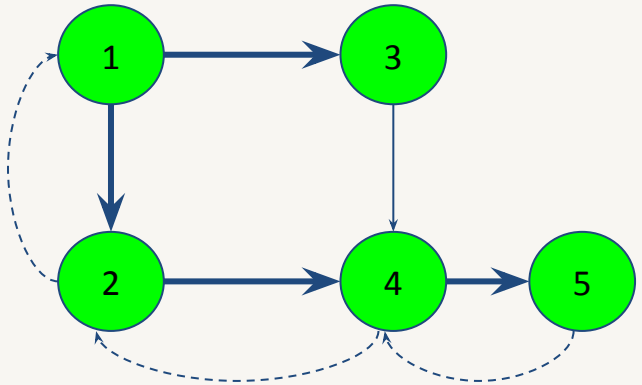
Start Time

1	0	2	3	4
1	2	3	4	5

End Time

0	7	0	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	1	1	1
1	2	3	4	5

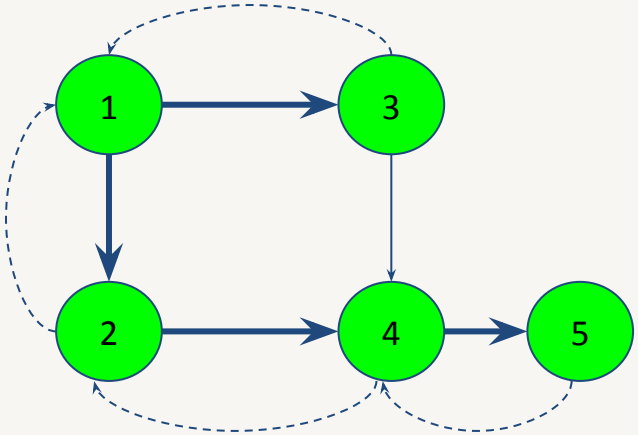
Start Time

1	8	2	3	4
1	2	3	4	5

End Time

0	7	0	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	1	1	1
1	2	3	4	5

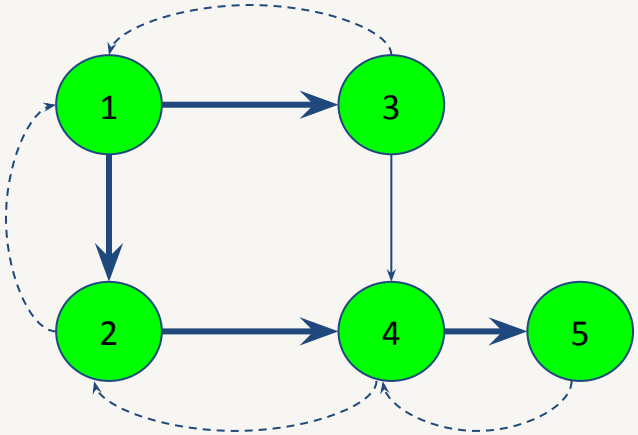
Start Time

1	8	2	3	4
1	2	3	4	5

End Time

0	7	9	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	1	1	1
1	2	3	4	5

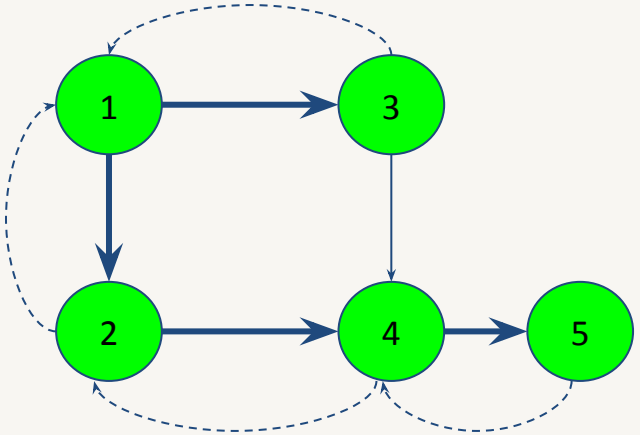
Start Time

1	8	2	3	4
1	2	3	4	5

End Time

10	7	9	6	5
1	2	3	4	5

Topological Sorting using DFS



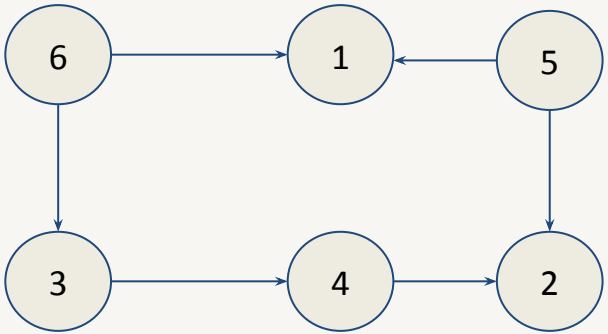
End Time

10	7	9	6	5
1	2	3	4	5

If we sort the ending times based on descending order, we get 10, 9, 7, 6, 5 i.e. vertices 1, 3, 2, 4, 5 which is a topological ordering for the above graph.

Topological Sorting using BFS

- Also known as Kahn's Algorithm for Topological Sorting.



Indegree

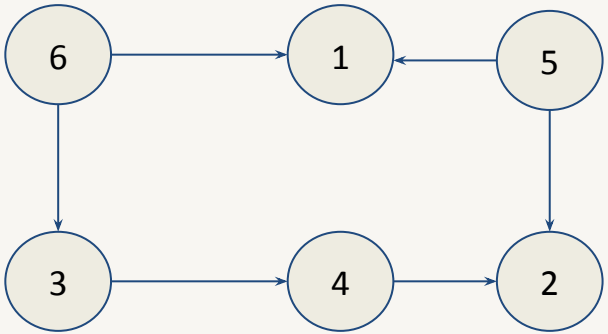
2	2	1	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering:

Topological Sorting using BFS

- Initially, enqueue all the nodes with indegree 0 into the queue i.e. 5 and 6 in this case.



Indegree

2	2	1	1	0	0
1	2	3	4	5	6

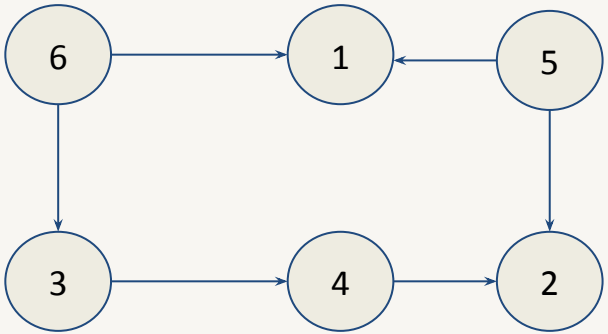
5 6

Queue

Topological Ordering:

Topological Sorting using BFS

- Dequeue from the front of the queue i.e. 5 in this case.



Indegree

2	2	1	1	0	0
1	2	3	4	5	6

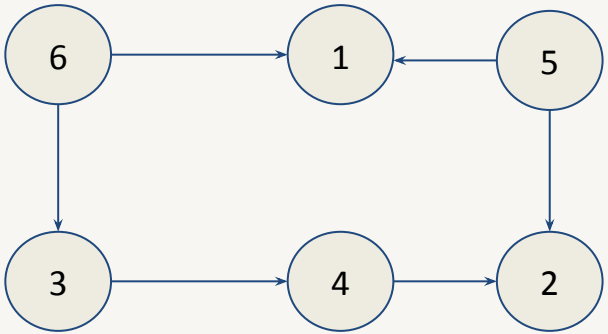
6

Queue

Topological Ordering:

Topological Sorting using BFS

- Now, enqueue from the front of the queue i.e. 5 in this case.



Indegree

2	2	1	1	0	0
1	2	3	4	5	6

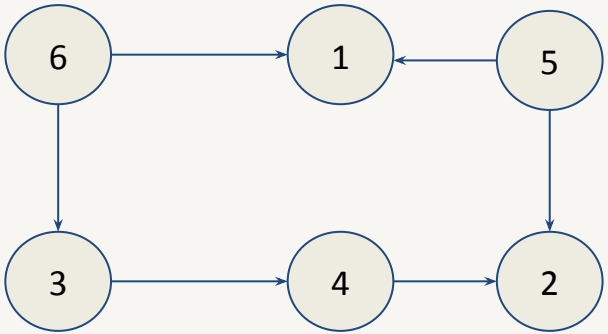
6

Queue

Topological Ordering:

Topological Sorting using BFS

- Now, reduce the indegree of the adjacent vertices of 5 i.e. 1 and 2.



Indegree

1	1	1	1	0	0
1	2	3	4	5	6

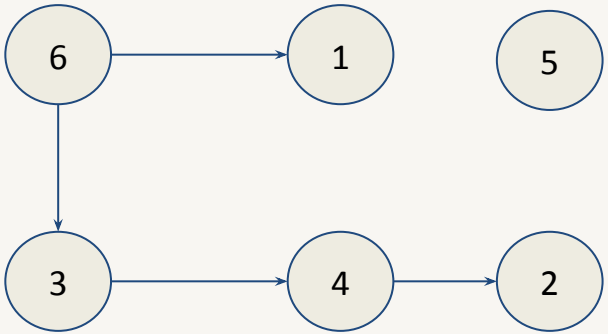
6

Queue

Topological Ordering: 5

Topological Sorting using BFS

- Next, remove from the front of the queue i.e. 6 in this case.



Indegree

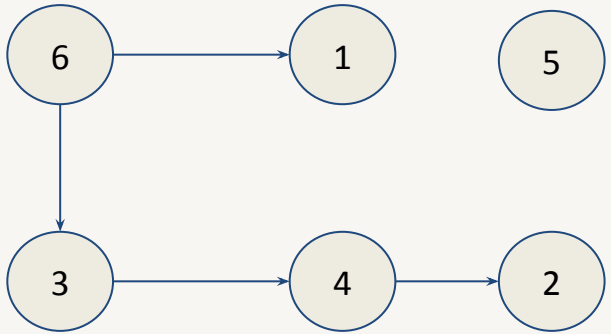
1	1	1	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5

Topological Sorting using BFS

- Now, reduce the indegree of the adjacent vertices of 6 i.e. 1 and 3.



Indegree

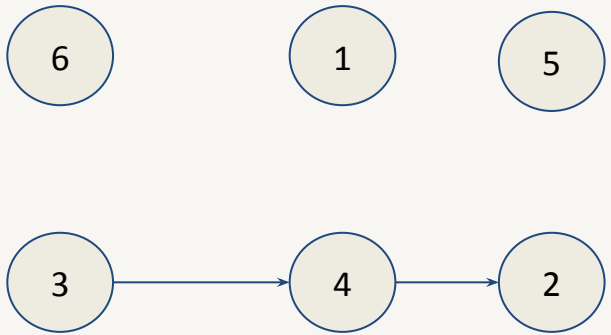
0	1	0	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6

Topological Sorting using BFS

- Indegree of vertices 1 and 3 is 0 now. Therefore, insert them into the queue.



Indegree

0	1	0	1	0	0
1	2	3	4	5	6

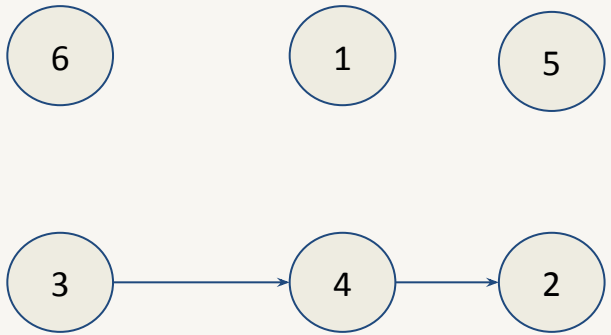
1 3

Queue

Topological Ordering: 5 6

Topological Sorting using BFS

- Remove from the front of queue i.e. 1. Now, 1 does not have any outgoing edges so go to next step and write 1 in the ordering.



Indegree

0	1	0	1	0	0
1	2	3	4	5	6

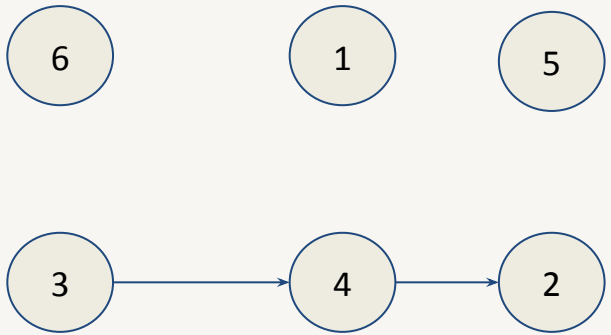
3

Queue

Topological Ordering: 5 6 1

Topological Sorting using BFS

- Again, remove from the front of queue i.e. 3.



Indegree

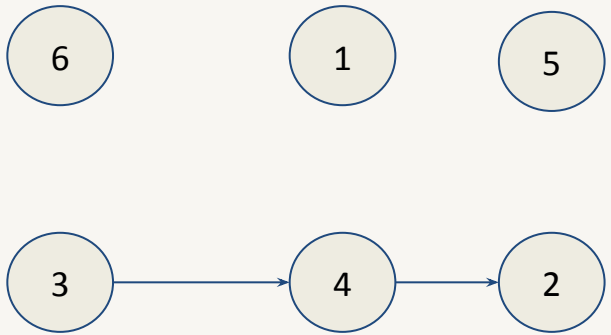
0	1	0	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6 1

Topological Sorting using BFS

- Reduce the indegree of adjacent vertices of 3 i.e. 4 in this case.



Indegree

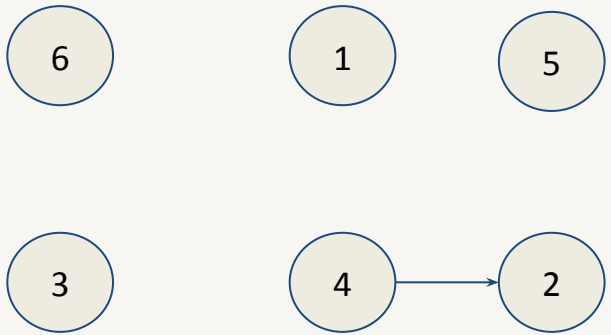
0	1	0	0	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6 1 3

Topological Sorting using BFS

- Indegree of vertex 4 is 0 now. Enqueue it into the queue.



Indegree

0	1	0	0	0	0
1	2	3	4	5	6

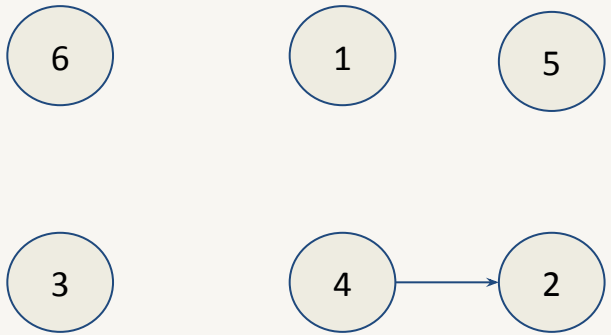
4

Queue

Topological Ordering: 5 6 1 3

Topological Sorting using BFS

- Dequeue from the queue i.e. 4. Reduce the indegree of adjacent vertices of 4 i.e. 2.



Indegree

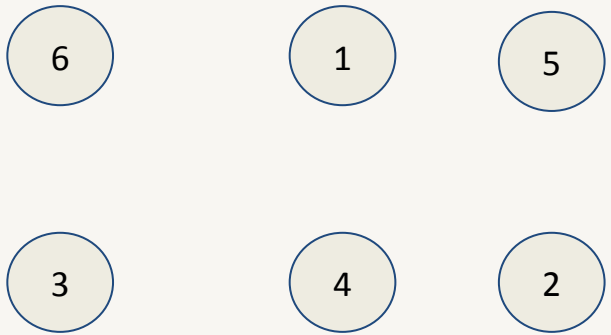
0	0	0	0	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6 1 3 4

Topological Sorting using BFS

- Enqueue 2 into the queue has it has indegree 0.



Indegree

0	0	0	0	0	0
1	2	3	4	5	6

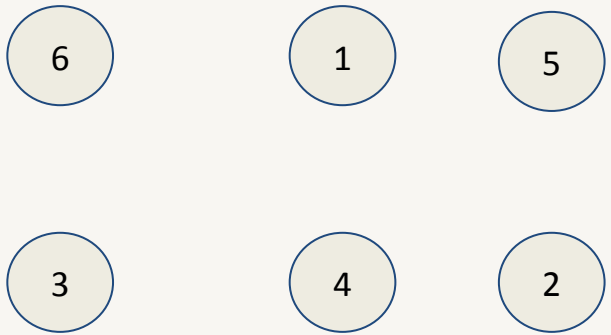
2

Queue

Topological Ordering: 5 6 1 3 4

Topological Sorting using BFS

- Dequeue from the front of queue, and since 2 has no adjacent vertices, write it in the ordering. 5 6 1 3 4 2 is topological sorting for the graph.



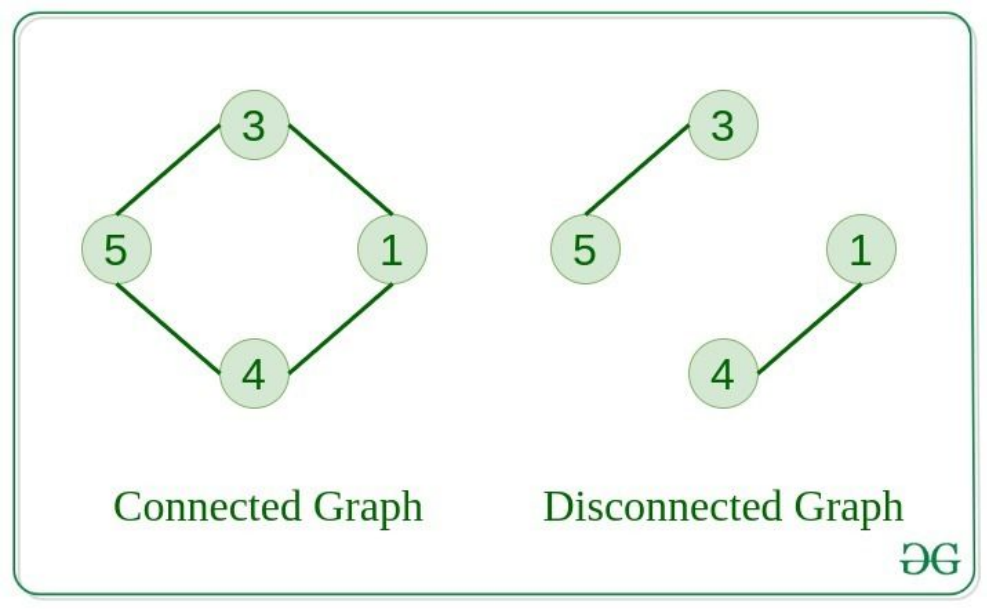
Indegree

0	0	0	0	0	0
1	2	3	4	5	6

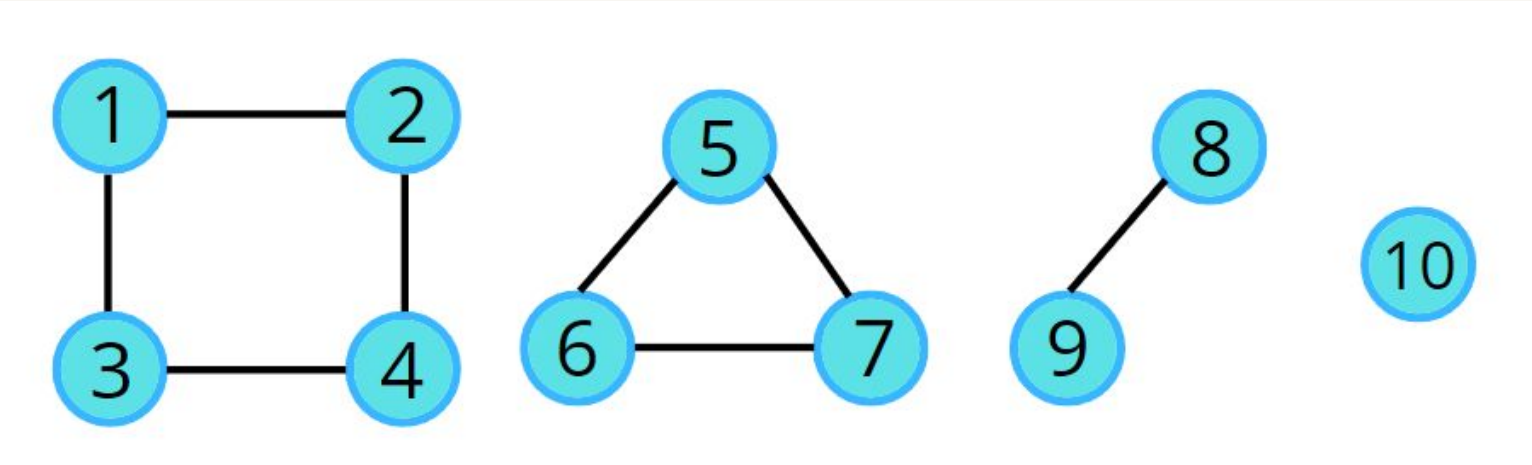
Queue

Topological Ordering: 5 6 1 3 4 2

Connected Components

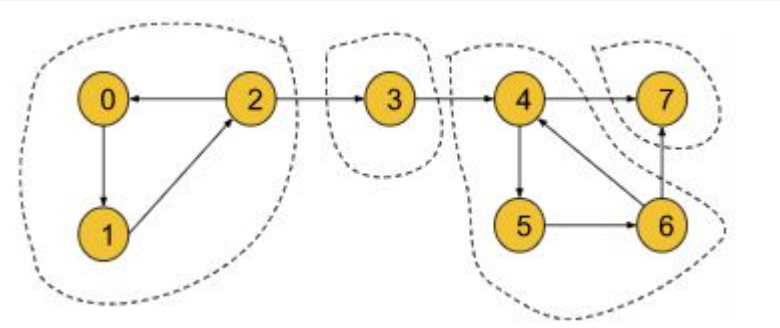


Connected Components



Strongly Connected Components

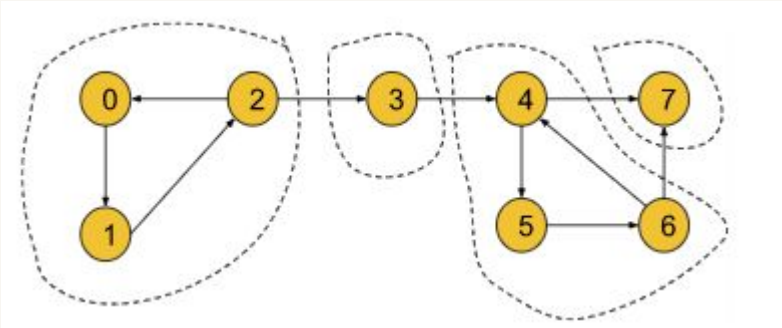
- A component is called a Strongly Connected Component(SCC) only if for every possible pair of vertices (u, v) inside that component, u is reachable from v and v is reachable from u.



In this graph, [0, 1, 2] is a strongly connected component. Here, we can reach from 0 to 1 and vice versa, 1 to 2 and vice versa and 2 to 0 and vice versa.

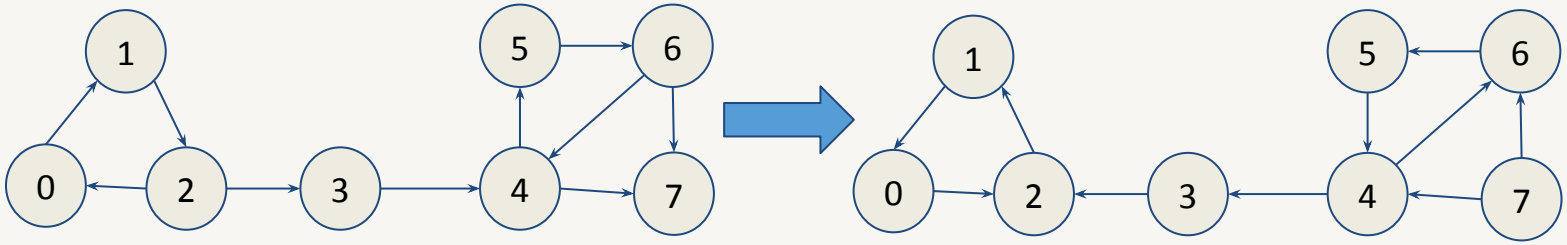
Strongly Connected Components

- Similarly, in this graph, there are 4 strongly connected components, [0, 1, 2], [3], [4, 5, 6] and 7.



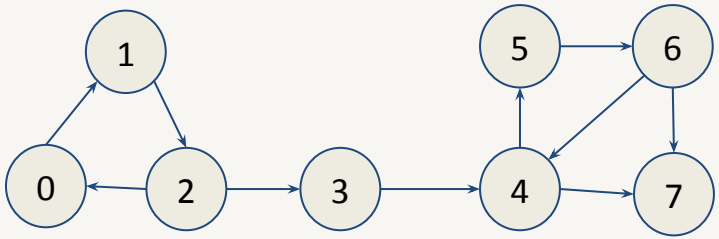
Kosaraju's Algorithm

- Initially, transpose the given graph i.e. edge (u, v) becomes (v, u).



Kosaraju's Algorithm

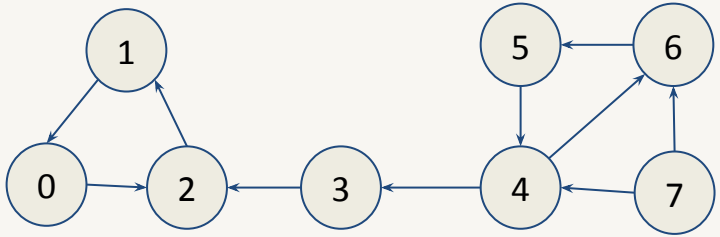
- Run DFS on the initial graph from the source vertex.



Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Stack



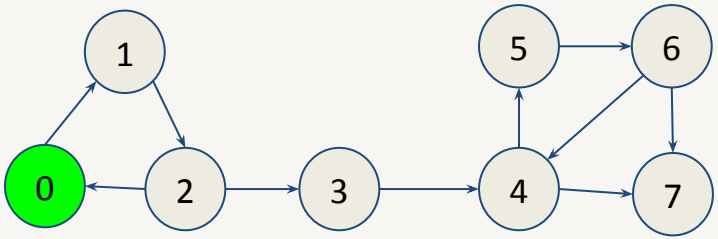
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex.

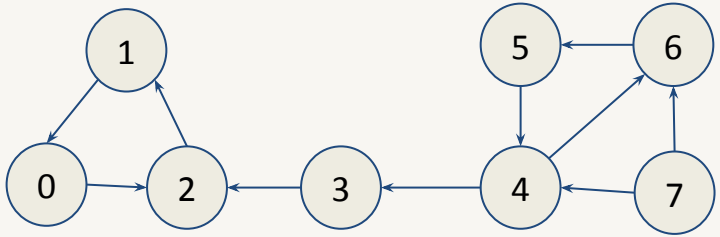


Visited

1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

0

Stack



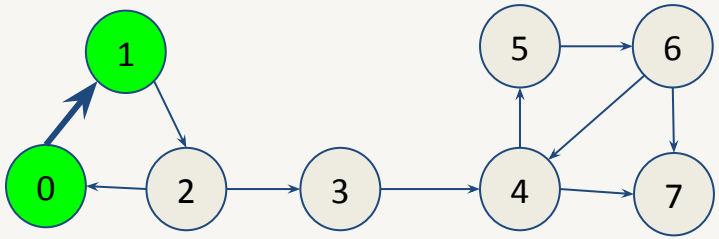
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex.

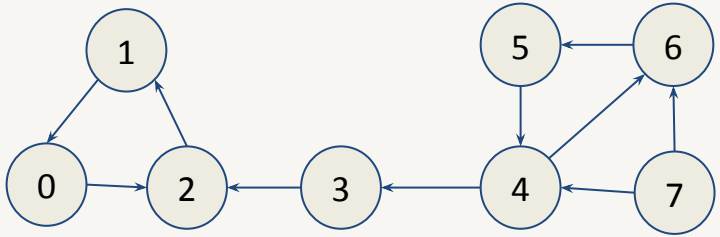


Visited

1	1	0	0	0	0	0	0
0	1	2	3	4	5	6	7

0 1

Stack



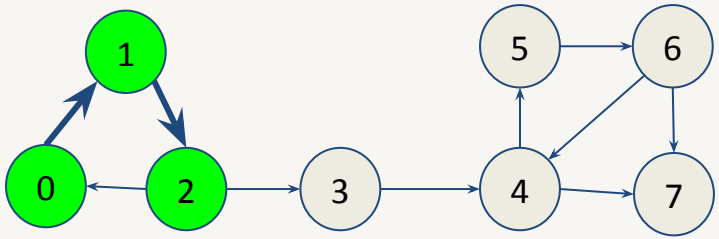
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex.

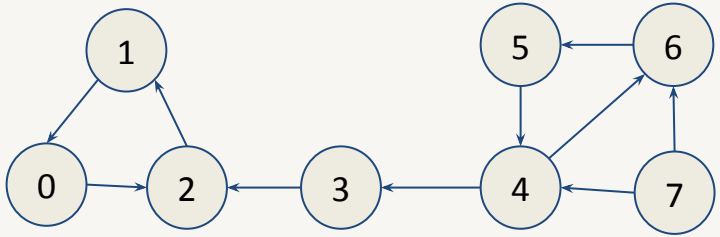


Visited

1	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7

0 1 2

Stack



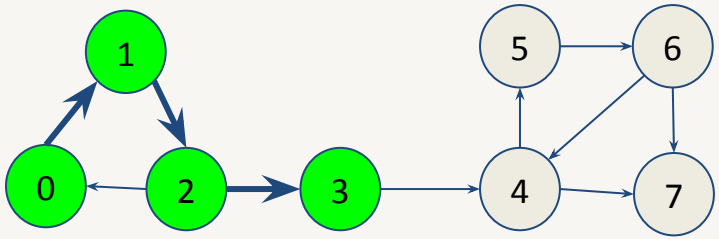
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex.

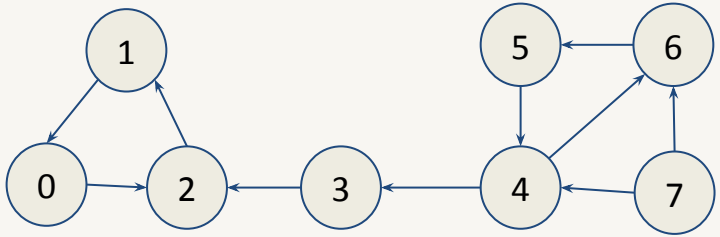


Visited

1	1	1	1	0	0	0	0
0	1	2	3	4	5	6	7

0 1 2 3

Stack



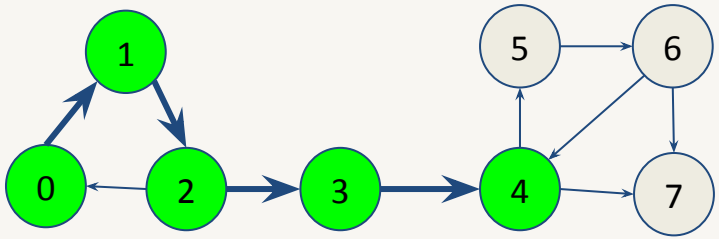
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex.

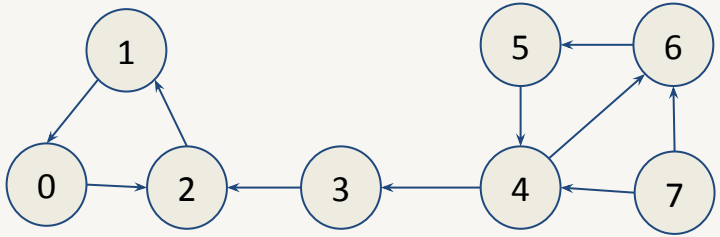


Visited

1	1	1	1	1	0	0	0
0	1	2	3	4	5	6	7

0 1 2 3 4

Stack



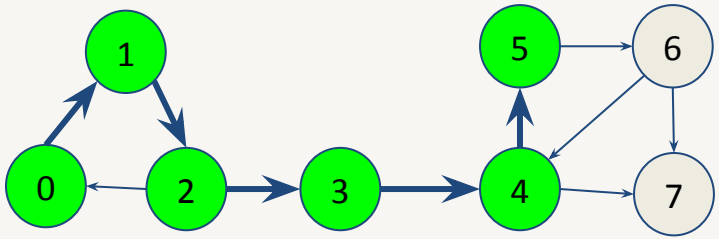
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex.

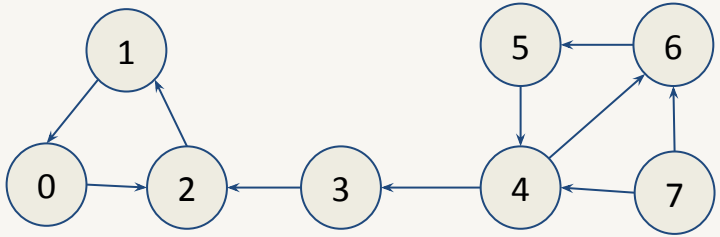


Visited

1	1	1	1	1	1	0	0
0	1	2	3	4	5	6	7

0 1 2 3 4 5

Stack



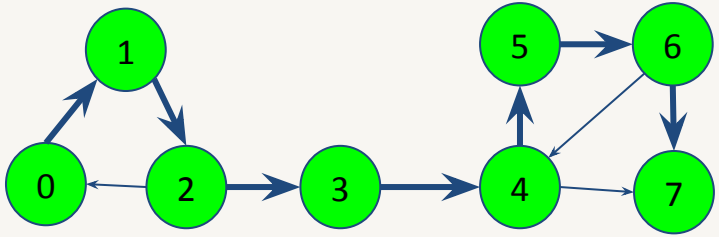
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex.

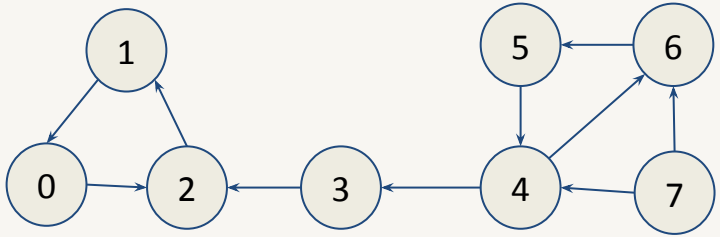


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0 1 2 3 4 5 6 7

Stack



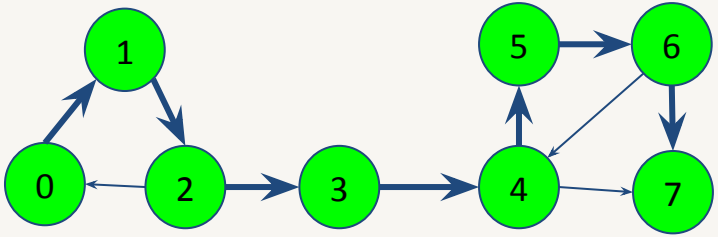
Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Pop all the vertices from the stack and push them into the reference stack.

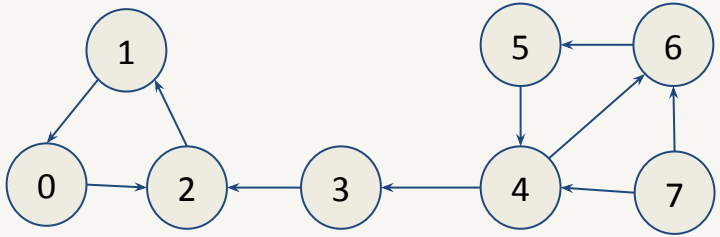


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0 1 2 3 4 5 6

Stack



Visited

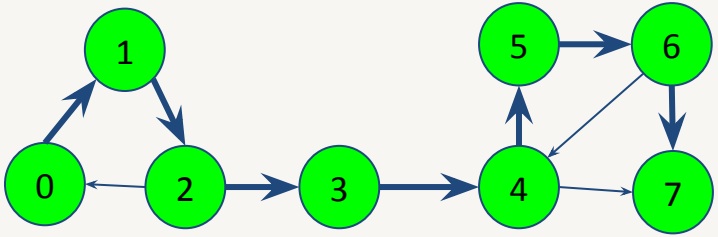
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7

Reference Stack

Kosaraju's Algorithm

- Pop all the vertices from the stack and push them into the reference stack.

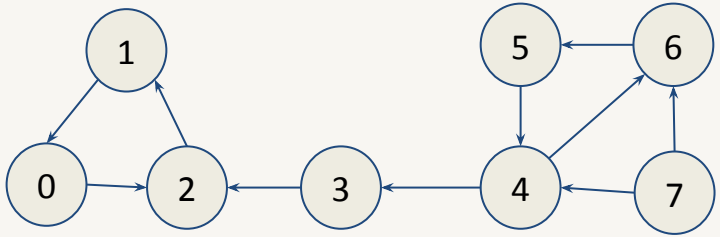


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0 1 2 3 4 5

Stack



Visited

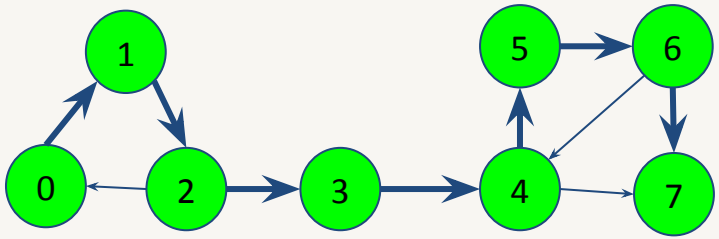
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7 6

Reference Stack

Kosaraju's Algorithm

- Pop all the vertices from the stack and push them into the reference stack.

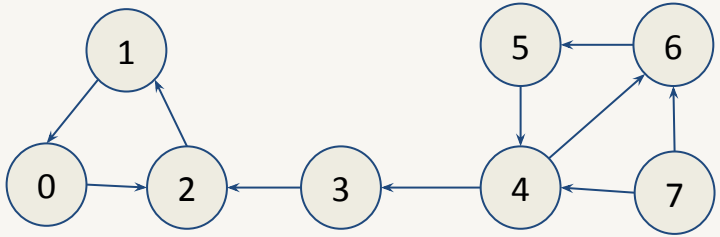


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0 1 2 3 4

Stack



Visited

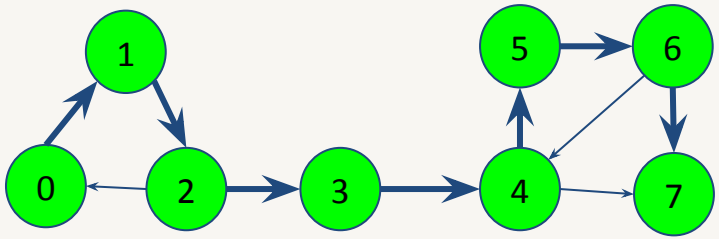
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7 6 5

Reference Stack

Kosaraju's Algorithm

- Pop all the vertices from the stack and push them into the reference stack.

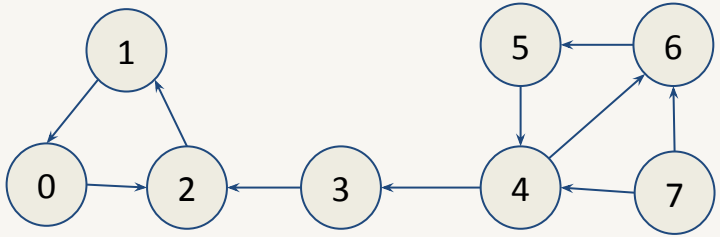


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0 1 2 3

Stack



Visited

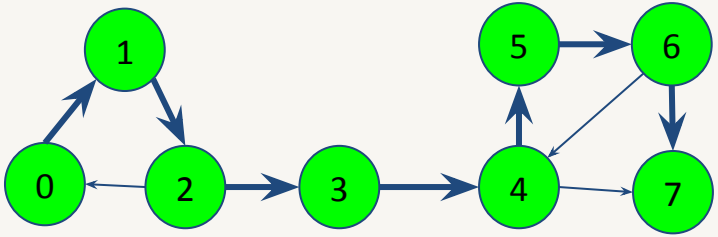
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7 6 5 4

Reference Stack

Kosaraju's Algorithm

- Pop all the vertices from the stack and push them into the reference stack.

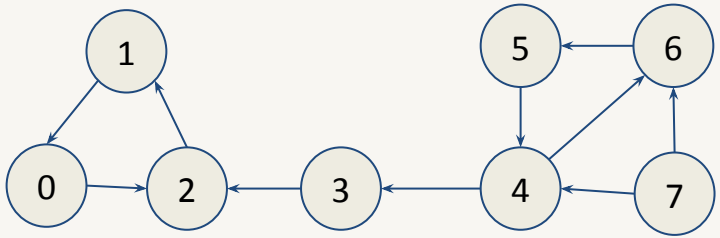


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0 1 2

Stack



Visited

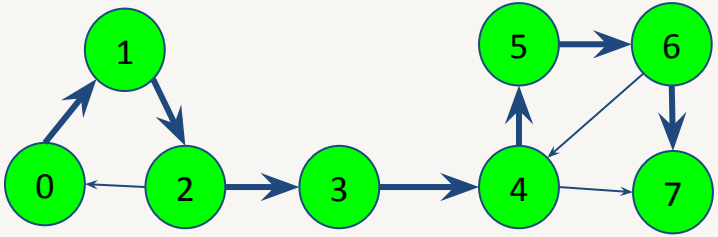
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7 6 5 4 3

Reference Stack

Kosaraju's Algorithm

- Pop all the vertices from the stack and push them into the reference stack.

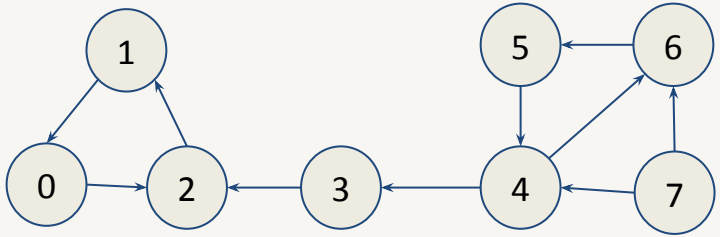


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0 1

Stack



Visited

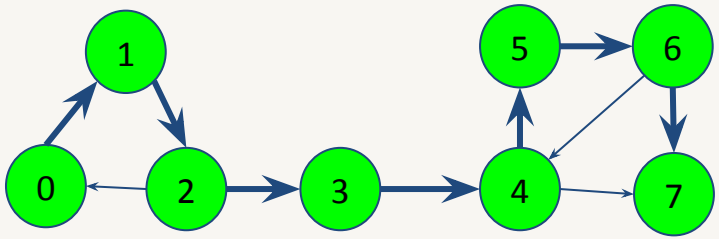
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7 6 5 4 3 2

Reference Stack

Kosaraju's Algorithm

- Pop all the vertices from the stack and push them into the reference stack.

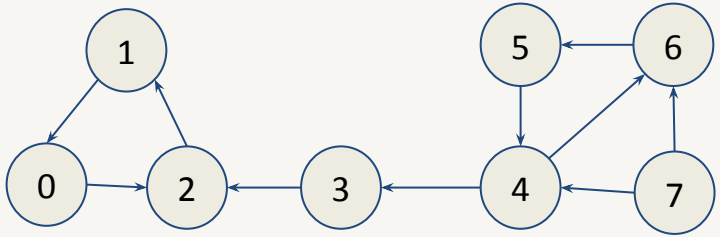


Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

0

Stack



Visited

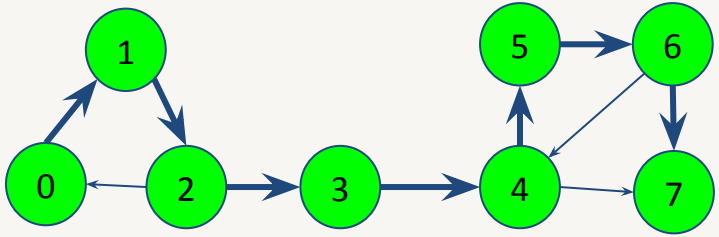
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7 6 5 4 3 2 1

Reference Stack

Kosaraju's Algorithm

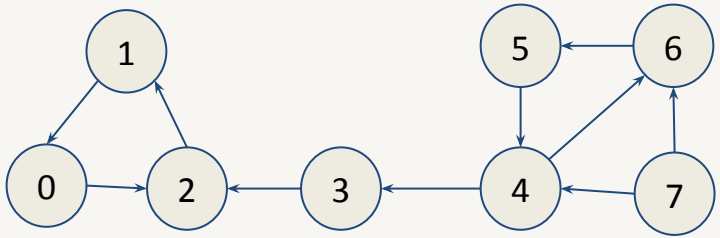
- Pop all the vertices from the stack and push them into the reference stack.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Stack



Visited

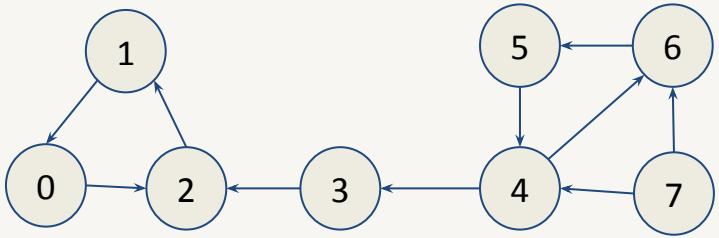
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7 6 5 4 3 2 1 0

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 0 in this case and run DFS starting from 0.



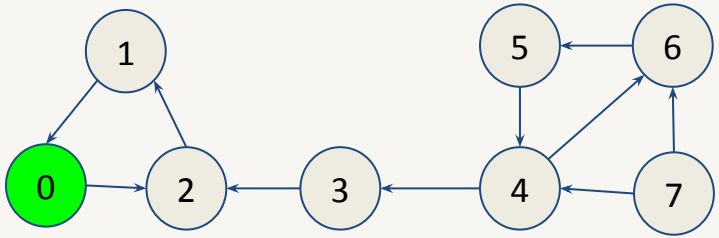
Visited

0	0	0	0	0	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5 4 3 2 1 0

Reference Stack

Kosaraju's Algorithm



DFS(0)

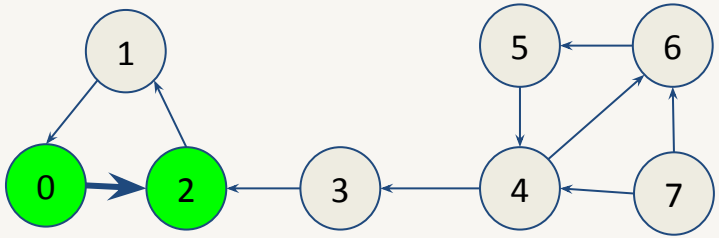
Visited

1	0	0	0	0	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5 4 3 2 1

Reference Stack

Kosaraju's Algorithm



DFS(0) -> DFS(2)

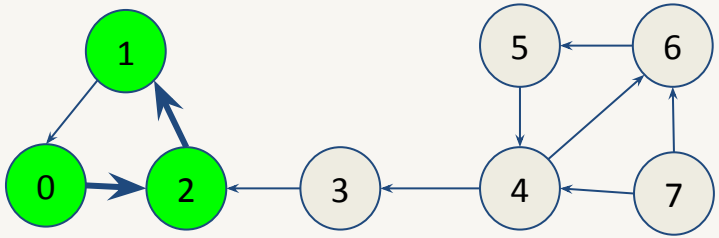
Visited

1	0	1	0	0	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5 4 3 2 1

Reference Stack

Kosaraju's Algorithm



DFS(0) -> DFS(2) -> DFS(1)

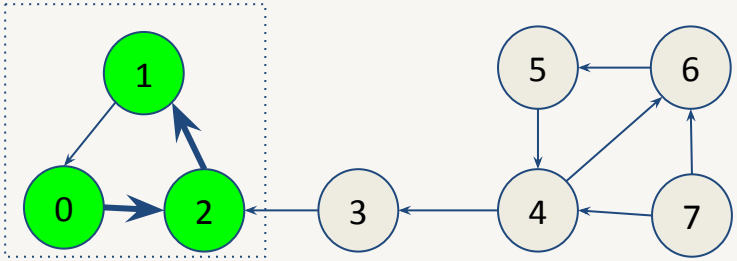
Visited

1	1	1	0	0	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5 4 3 2 1

Reference Stack

Kosaraju's Algorithm



Visited

1	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4	3	2	1

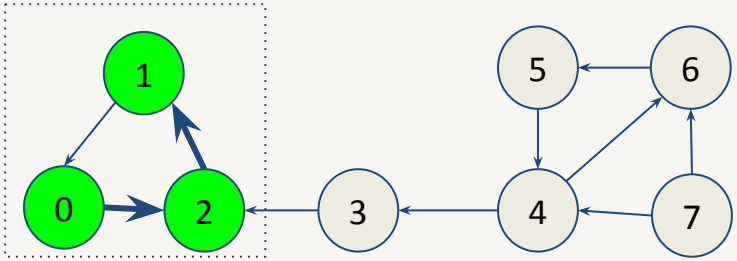
Reference Stack

DFS(0) -> DFS(2) -> DFS(1)

Now, from 1 we can go to 0 however it is already visited. Since there are no unvisited vertex, we'll backtrack to 2 and from 2, we'll backtrack to 0. Therefore, [0, 2, 1] is our first SCC.

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 1 in this case. However, 1 is already visited.



Visited

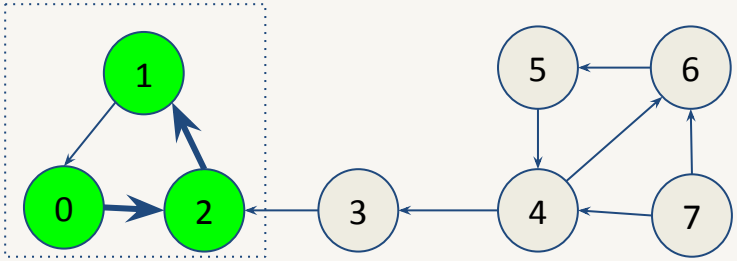
1	1	1	0	0	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5 4 3 2

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 2 in this case. However, 2 is already visited.



Visited

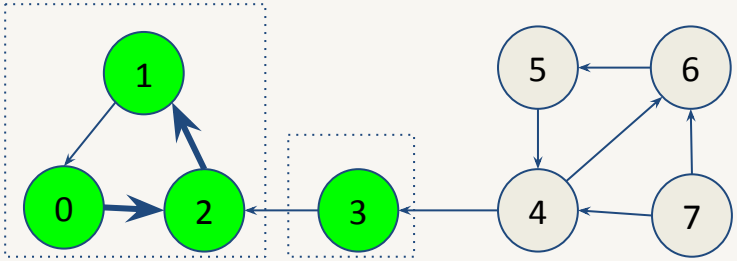
1	1	1	0	0	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5 4 3

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 3 in this case. Run DFS from 3.



Visited

1	1	1	1	0	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5 4

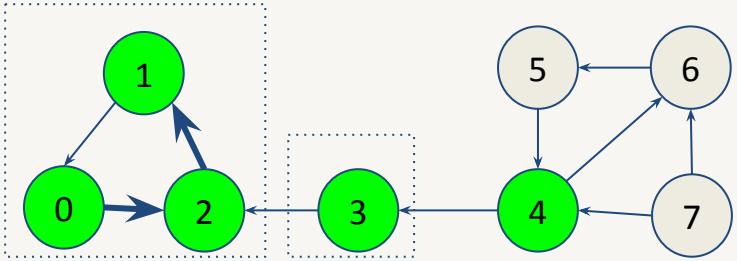
Reference Stack

DFS(3)

From 3, we can go to 1. However, 1 is already visited. As we don't have any other adjacent vertices, 3 is our second SCC.

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 4 in this case. Run DFS from 4.



DFS(4)

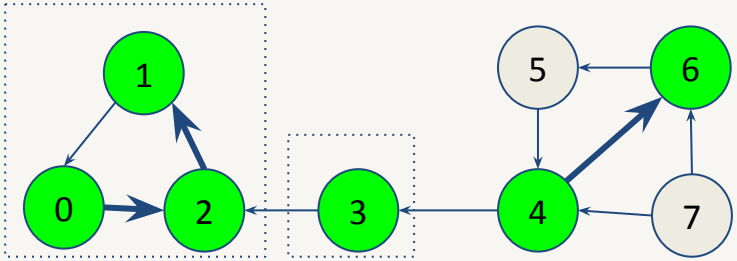
Visited

1	1	1	1	1	0	0	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5

Reference Stack

Kosaraju's Algorithm



DFS(4) -> DFS(6)

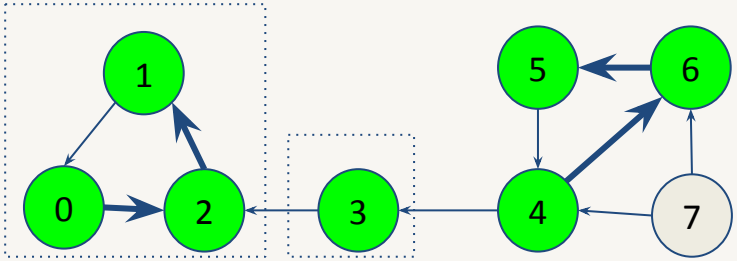
Visited

1	1	1	1	1	0	1	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5

Reference Stack

Kosaraju's Algorithm



DFS(4) -> DFS(6) -> DFS(5)

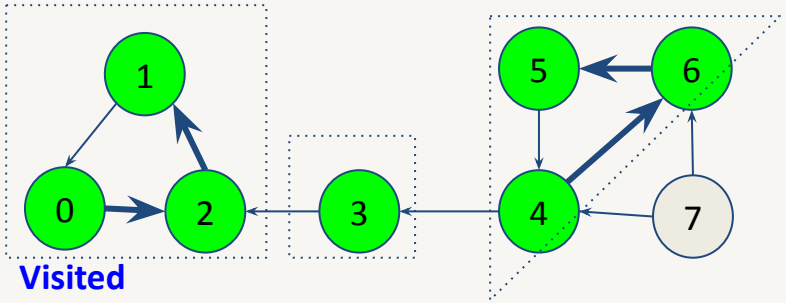
Visited

1	1	1	1	1	1	1	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7 6 5

Reference Stack

Kosaraju's Algorithm



Visited

1	1	1	1	1	1	1	0
0	1	2	3	4	5	6	7

7 6 5

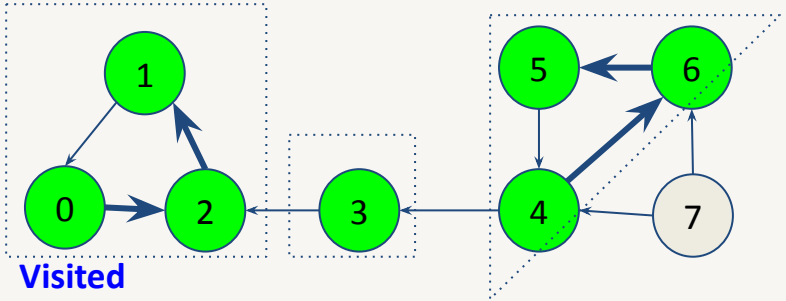
Reference Stack

DFS(4) -> DFS(6) -> DFS(5)

From 5 we can go to 4. However, it is already visited. So, we'll backtrack from 5 to 6, and then 6 to 4. Since 4 doesn't any adjacent vertices, therefore, [4, 6, 5] is our third SCC.

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 5 in this case. However, 5 is already visited.



Visited

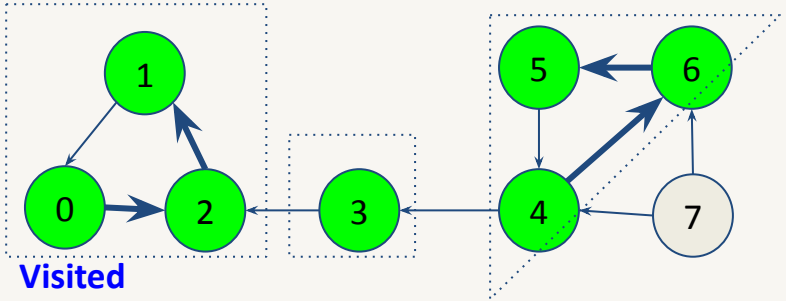
1	1	1	1	1	1	1	0
0	1	2	3	4	5	6	7

7 6

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 6 in this case. However, 6 is already visited.



Visited

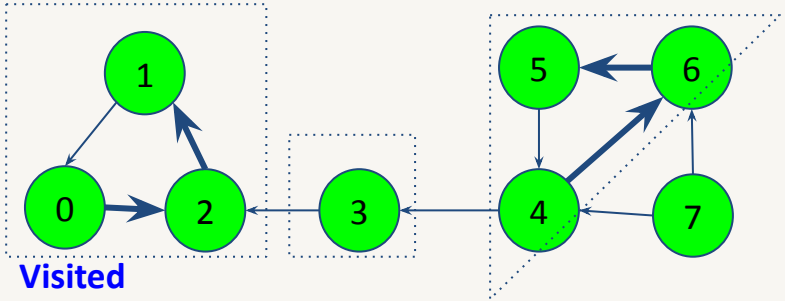
1	1	1	1	1	1	1	0
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

7

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 7 in this case. Run DFS from 7.



DFS(7)

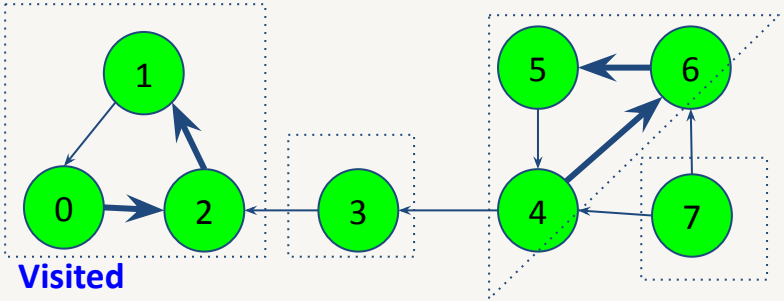
Visited

1	1	1	1	1	1	1	1
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 7 in this case. Run DFS from 7.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Reference Stack

DFS(7)

7 has two adjacent vertices 4 and 6, however, both are already visited. Therefore, 7 is our fourth SCC.

[0, 2, 1]: SCC1
[3]: SCC2
[4, 6, 5]: SCC3
[7]: SCC4

This is Kosaraju's Algorithm!

References

- Mumit sir’s slide - https://drive.google.com/file/d/1OkLO91GV4M23e-1EtvFr9yeGWgk6V91q/view

- PNP sir’s slide - https://docs.google.com/presentation/d/1bKKT_3Mnlp0cV509k7E8hBWEKQ_YfV0cfek10LDCo6Y/edit?slide=id.g26e6291b7d8_0_183#slide=id.g26e6291b7d8_0_183